



CPE241 Database System Project Report

Project Title:

ModLearn

Submitted by

Wisit Suwannao	67070501042
Guntee DOUNGmanee	67070501003
Natthawat Primsirikunawut	67070501027
Supawit Marayat	67070501045
Polwarit Watthanahemmarat	67070501067
Panuwat Boonsak	67070501034
Rattananan Siriponvat	67070501038

Submitted to

Dr. Jaturon Harnsomburana

Dr. Piyanit Ua-areemitr

Department of Computer Engineering
King Mongkut's University of Technology Thonburi
Semester 2025/2

Project Title: ModLearn

Instructor: Dr. Jaturon Harnsomburana,
Dr. Piyanit Ua-areemitr

Course: CPE241 Database Systems

Semester: 2025/2

Date: May 13, 2025

Team member:

Student ID	Name
67070501042	Wisit Suwannao
67070501003	Guntee Doungmanee
67070501027	Natthawat Primsirikunawut
67070501045	Supawit Marayat
67070501067	Polwarit Watthanahemmarat
67070501034	Panuwat Boonsak
67070501038	Rattananan Siriponvat

Contents

1	Requirement	3
1.1	Requirement Summary	3
1.2	Stakeholders and Scope	3
1.3	Core Business Rules	3
2	System Design	4
2.1	Functional Requirement	4
2.2	Non-functional requirements	9
2.3	Database Design	10
2.3.1	Entity Relationship Diagram (ERD)	10
2.3.2	Relationship Description	11
2.3.3	Attributes, Keys, and Constraints	15
2.3.4	Data Dictionary	17
2.3.5	Index Design	26
2.3.6	Referential Integrity	28
2.4	Query Design	29
2.4.1	Scope and Documentation Format	29
2.4.2	Category Queries	29
2.4.3	Course Catalog and Course Administration Queries	31
2.4.4	Commerce and Pricing Queries	42
2.4.5	Library and Access Queries	47
2.4.6	Watch Progress Queries	48
2.4.7	Review and Moderation Queries	52
2.4.8	Recommendation Queries	56
2.4.9	Analytics Queries	58
2.4.10	File and Media Queries	62
2.5	Optimization	63
2.5.1	Optimization Goals and Experimental Setup	63
2.5.2	Experiment 1: Public Course Catalog Query	64
2.5.3	Experiment 2: Continue Watching and Resume Query	65
2.5.4	Experiment 3: Review Listing and Rating Summary	66
2.5.5	Experiment 4: Analytics Aggregation	67
3	Application Implementation	69
3.1	Tech Stack	69
3.2	System Design	69
3.3	User Interface	69
4	Conclusion	71

1 Requirement

1.1 Requirement Summary

ModLearn is a web-based digital learning platform intended to support the discovery, acquisition, access, and continued consumption of structured online course content. The platform is expected to serve both public users and authenticated learners while also providing controlled administrative capabilities for maintaining the course catalog, lessons, classifications, pricing, reviews, and reporting data.

From the learner perspective, the system is expected to provide a complete learning-access flow. Users should be able to discover available courses, review course information, obtain access through valid ownership, continue lessons from previous progress, and receive discovery support through recommendation and continue-watching features. The system is therefore not limited to one-time page access; it must support continuity across repeated visits and sessions.

From the operational perspective, the platform must enable authorized administrators to manage the structure and release state of learning content. This includes maintaining courses, lessons, categories, associated media assets, monetization settings, moderation actions, and analytics views. Administrative behavior must remain separated from public or learner behavior through role-based access control and controlled access boundaries.

1.2 Stakeholders and Scope

Stakeholder	Primary concern
Public visitor	Discover visible courses and inspect public course information
Learner	Access owned content, resume progress, and interact with course feedback features
Administrator	Maintain content, pricing, classifications, media, moderation, and analytics
System owner	Ensure the platform supports controlled delivery, monetization, and operational oversight

The platform also depends on explicit business rules that govern ownership, payment-gated access, review eligibility, and lesson sequencing. These rules define what the domain allows, while later sections describe how the implemented schema constraints and application query flows enforce them.

1.3 Core Business Rules

The following rules summarize the core domain policies that ModLearn must preserve. They are stated in business terms here, while later sections connect them to database constraints and runtime service behavior.

Rule ID	Business rule	Business meaning
BR-01	One user can review one course only once.	A learner may maintain one current review per course rather than multiple parallel reviews for the same ownership relationship.
BR-02	One course cannot have duplicate lesson order values.	Each lesson in a course must occupy one distinct position in the learning sequence.
BR-03	Access is granted only after successful payment.	Owned-library access must not be granted from a pending or failed commerce event.
BR-04	One user cannot have duplicate ownership for the same course.	The same learner cannot hold duplicate ownership rows for a single course.

Rule ID	Business rule	Business meaning
BR-05	One user can have at most one active progress row per lesson.	Resume and completion state must remain singular for one learner and one lesson.
BR-06	One course-category assignment pair cannot be duplicated.	A course may belong to many categories, but the same category should not be attached twice to the same course.
BR-07	Public reviews are shown only for published and available courses.	Feedback for unreleased or unavailable courses must not be exposed through public review responses.
BR-08	Only users with active ownership can create or update a review.	Review submission is limited to legitimate learners with current access to the course.

2 System Design

2.1 Functional Requirement

Functional requirements describe what the system must do. They define the features, services, and behaviors that ModLearn must provide for public users, learners, and administrators.

Group	Scope
FR-01	User authentication and profile access
FR-02	Course catalog and public browsing
FR-03	Category and classification management
FR-04	Course, lesson, and media administration
FR-05	Purchase and owned-library access control
FR-06	Watch progress and continue-watching flow
FR-07	Ratings, reviews, and review moderation
FR-08	Personalized discovery and analytics

ID	Requirement Statement	Rationale	Verify
FR-01 User Authentication and Profile Access			
FR-01.1	The system shall allow a new user to create an account using email and password credentials.	Establishes a primary onboarding path.	Test
FR-01.2	The system shall authenticate a registered user before granting access to protected features.	Protects user-specific resources.	Test
FR-01.3	The system shall allow an authenticated user to sign out from an active session.	Supports session control and privacy.	Test

ID	Requirement Statement	Rationale	Verify
FR-01.4	The system shall maintain authenticated sessions using secure, server-managed session state.	Preserves login continuity across requests.	Inspection and test
FR-01.5	The system shall support username, display name, and profile image data for each user account.	Provides basic identity management.	Inspection and test
FR-01.6	The system shall restrict admin-only operations to users assigned the admin role.	Enforces role-based authorization.	Test
FR-02 Course Catalog and Public Browsing			
FR-02.1	The system shall present a public list of courses that are both published and available.	Ensures only releasable content is visible.	Test
FR-02.2	The system shall provide a public course detail view for a course that is both published and available.	Supports course discovery before access.	Test
FR-02.3	The system shall provide a public list of lessons belonging to a published and available course.	Allows users to inspect course structure.	Test
FR-02.4	The system shall support filtering or searching course listings using request parameters supported by the catalog API.	Enables scalable content discovery.	Test
FR-02.5	The system shall provide a public listing of popular courses based on recorded usage data.	Improves content discovery.	Analysis and test
FR-03 Category and Classification Management			
FR-03.1	The system shall provide a public list of categories available for browsing.	Supports structured navigation.	Test
FR-03.2	The system shall allow an administrator to create a category record.	Supports content organization.	Test
FR-03.3	The system shall allow an administrator to update mutable category data.	Supports maintenance of taxonomy data.	Test
FR-03.4	The system shall allow an administrator to delete a category record.	Supports taxonomy cleanup.	Test

ID	Requirement Statement	Rationale	Verify
FR-03.5	The system shall allow an administrator to assign one or more categories to a course.	Connects catalog browsing to content structure.	Test
FR-04 Course, Lesson, and Media Administration			
FR-04.1	The system shall allow an administrator to create a course container.	Establishes managed learning content.	Test
FR-04.2	The system shall allow an administrator to update course metadata, including title and descriptive fields.	Supports content maintenance.	Test
FR-04.3	The system shall allow an administrator to set the publish state of a course.	Separates draft and released content.	Test
FR-04.4	The system shall allow an administrator to set the availability state of a course independently from publish state.	Supports release and access control rules.	Test
FR-04.5	The system shall allow an administrator to add lessons to a course.	Supports multi-lesson course delivery.	Test
FR-04.6	The system shall allow an administrator to update lesson metadata.	Supports lesson maintenance.	Test
FR-04.7	The system shall allow an administrator to reorder lessons within a course while preserving a unique lesson-order position for each lesson.	Preserves intended learning sequence.	Test
FR-04.8	The system shall allow an administrator to remove a lesson from a course.	Supports content lifecycle management.	Test
FR-04.9	The system shall allow an administrator to generate signed upload requests for media files.	Supports controlled media ingestion.	Test
FR-04.10	The system shall allow an administrator to generate signed download URLs for stored files.	Supports controlled media retrieval.	Test
FR-04.11	The system shall allow an administrator to delete file metadata and backing object references.	Supports storage governance.	Test
FR-05 Purchase and Owned-Library Access Control			

ID	Requirement Statement	Rationale	Verify
FR-05.1	The system shall allow an authenticated user to purchase an eligible course.	Provides a monetization path.	Test
FR-05.2	The system shall grant owned-library access to a purchased course only after payment is recorded as successful.	Prevents unauthorized entitlement.	Test
FR-05.3	The system shall provide the signed-in user with a list of owned library items.	Supports post-purchase access.	Test
FR-05.4	The system shall provide the signed-in user with owned course details for a course in the user library.	Supports direct consumption of owned content.	Test
FR-05.5	The system shall verify whether the signed-in user has valid ownership for a requested course or lesson before protected delivery.	Enforces entitlement rules.	Test
FR-05.6	The system shall allow authorized pricing administrators to create and update course pricing windows.	Supports controlled pricing changes.	Test
FR-06 Watch Progress and Continue-Watching Flow			
FR-06.1	The system shall save watch-progress state for an authenticated user and lesson.	Enables resume behavior.	Test
FR-06.2	The system shall allow an authenticated user to mark lesson progress as completed.	Supports completion tracking.	Test
FR-06.3	The system shall return the saved resume position for a requested lesson when progress exists.	Supports seamless continuation.	Test
FR-06.4	The system shall return the next resume target within a course for an authenticated user.	Supports course continuation.	Test
FR-06.5	The system shall return the next auto-play lesson candidate within a course when available.	Supports continuous viewing flow.	Test
FR-06.6	The system shall provide a continue-watching list for the signed-in user.	Improves return-user experience.	Test
FR-07 Ratings, Reviews, and Review Moderation			

ID	Requirement Statement	Rationale	Verify
FR-07.1	The system shall provide public review listings for a course only when the course is published and available.	Prevents exposure of unreleased content feedback.	Test
FR-07.2	The system shall provide public review summary metrics for a course.	Supports trust and discovery.	Test
FR-07.3	The system shall allow an authenticated user with active ownership to create or update exactly one review per course.	Restricts reviews to legitimate learners.	Test
FR-07.4	The system shall allow an authenticated user to delete the user's own review.	Supports user control over submitted feedback.	Test
FR-07.5	The system shall allow an administrator to list reviews across moderation states.	Supports review governance.	Test
FR-07.6	The system shall allow an administrator to hide a review from public responses.	Supports moderation actions.	Test
FR-07.7	The system shall allow an administrator to restore a hidden review to public visibility.	Supports reversible moderation.	Test
FR-07.8	The system shall allow an administrator to permanently delete a review.	Supports policy enforcement.	Test
FR-08 Personalized Discovery and Analytics			
FR-08.1	The system shall provide personalized course recommendations for an authenticated user.	Improves relevance of discovery.	Test
FR-08.2	The system shall provide popular recommendation candidates for an authenticated user.	Supports discovery using aggregate demand.	Analysis and test
FR-08.3	The system shall provide recently added recommendation candidates for an authenticated user.	Supports freshness-based discovery.	Test
FR-08.4	The system shall provide administrators with overview analytics for a selected reporting window.	Supports operational monitoring.	Test
FR-08.5	The system shall provide administrators with per-course performance analytics.	Supports content evaluation.	Test

ID	Requirement Statement	Rationale	Verify
FR-08.6	The system shall provide administrators with lesson engagement analytics, including completion and drop-off indicators.	Supports learning-flow analysis.	Test
FR-08.7	The system shall provide administrators with lesson view analytics and view-session analytics.	Supports usage analysis and reporting.	Test

2.2 Non-functional requirements

The non-functional requirements of ModLearn define the quality attributes that the system must satisfy in addition to its functional behavior.

ID	Requirement Statement	Verify
NFR-01	The system shall protect protected learner and administrator operations through authenticated session handling and role-based authorization.	Inspection and test
NFR-02	The system shall validate request inputs and response structures consistently at the application boundary to reduce invalid or unsafe data exchange.	Inspection and test
NFR-03	The system shall restrict cross-origin access according to configured server policy and shall allow only approved origins, headers, and methods.	Inspection and test
NFR-04	The system shall provide controlled file access through signed upload and download operations rather than unrestricted direct storage access.	Inspection and test
NFR-05	The system shall maintain data integrity through relational constraints, unique keys, foreign keys, and domain validation rules within the database schema.	Inspection and test
NFR-06	The system shall support continuity and acceptable response behavior for repeated learning activity such as course browsing, lesson resume, and continue-watching flows.	Test
NFR-07	The system shall remain usable across common desktop and mobile screen sizes through a responsive web interface.	Test
NFR-08	The system shall be maintainable through modular separation of concerns across authentication, routing, service logic, validation, and persistence layers.	Inspection
NFR-09	The system shall support extensibility so that additional modules, routes, and data features can be added without redesigning the entire platform structure.	Inspection
NFR-10	The system shall provide implementation-level testability through automated tests for core modules, routers, services, and storage behavior.	Inspection and test
NFR-11	The system shall provide API discoverability and integration support through generated OpenAPI documentation for exposed service routes.	Inspection and test

ID	Requirement Statement	Verify
NFR-12	The system shall preserve operational availability by initializing required backend dependencies, including storage resources, before normal media operations are performed.	Inspection and test

2.3 Database Design

In this project, the database uses a relational PostgreSQL schema implemented through Drizzle ORM. It stores authentication, course, media, commerce, progress, analytics, and review data needed by the ModLearn platform.

2.3.1 Entity Relationship Diagram (ERD)

The full ERD shows how the schema is divided into authentication, course management, commerce, media, progress tracking, analytics, and review moderation. The placeholder below is reserved for the complete diagram of the implemented tables.

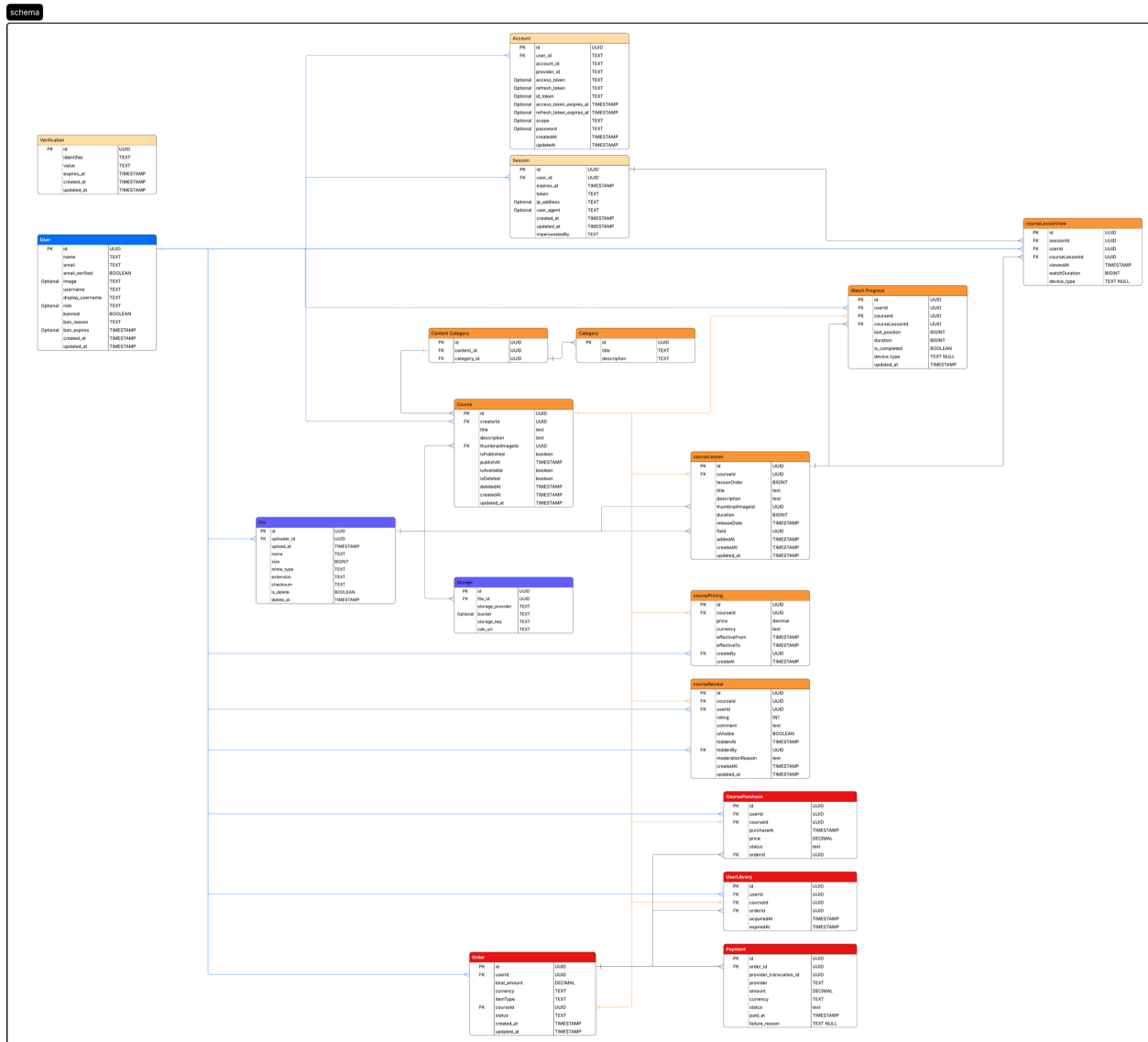


Figure 2.3.0 Complete ModLearn entity relationship diagram

2.3.2 Relationship Description

This subsection summarizes the implemented schema relationships defined by the foreign keys in the server-side database schema. It covers core business relationships and also includes optional or technical links that affect ownership, media attachment, moderation, and analytics tracking.

Authentication and identity

1. **User and Session**: One user can have many sessions, while each session belongs to exactly one user through `session.userId`.

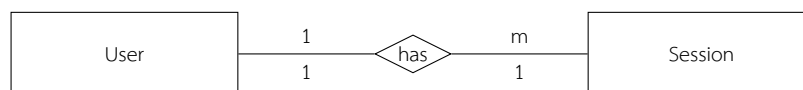


Figure 2.3.1 Relationship between **User** and **Session**

2. **User and Account:** One user can link many accounts, while each linked account belongs to exactly one user through `account.userId`.

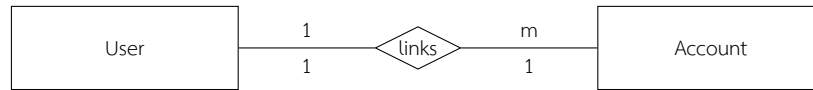


Figure 2.3.2 Relationship between **User** and **Account**

3. **User and File:** One user can upload many file metadata rows, while each **File** row belongs to exactly one user through `file.uploaderId`.

Course content and categorization

4. **User and Course:** One user can create many courses, while each course is created by exactly one user through `course.creatorId`.



Figure 2.3.3 Relationship between **User** and **Course**

4. **Course and CourseLesson:** One course can contain many lessons, while each lesson belongs to exactly one course through `course_lesson.courseId`. This relationship implements the business rule that one course cannot contain duplicate lesson order values, so lesson order is additionally constrained to be unique within the same course.



Figure 2.3.4 Relationship between **Course** and **CourseLesson**

5. **Course, Category, and CourseCategory:** Courses and categories form a many-to-many relationship implemented through the junction table **CourseCategory**. A course can be assigned to many categories, and a category can classify many courses.



Figure 2.3.5 Relationship between **Course**, **CourseCategory**, and **Category**

7. **Course and File (thumbnail):** A course may optionally reference one thumbnail file through `course.thumbnailImageId`, while one **File** row can be reused by many courses as a thumbnail if the application allows it.
8. **CourseLesson and File (thumbnail):** A lesson may optionally reference one thumbnail file through `course_lesson.thumbnailImageId`, while one **File** row can be reused by many lessons as a thumbnail.
9. **CourseLesson and File (lesson media):** A lesson may optionally reference one media file through `course_lesson.fileId`, while one **File** row can be referenced by many lessons if the same uploaded asset is reused.

Commerce and ownership

10. **User and Order:** One user can place many orders, while each order belongs to exactly one user through `order.userId`.



Figure 2.3.6 Relationship between **User** and **Order**

11. **Order and Payment:** One order can have many payment records, while each payment record belongs to exactly one order through `payment.orderId`. This allows the system to record retries or multiple provider-side payment attempts.



Figure 2.3.7 Relationship between **Order** and **Payment**

12. **Order and Course:** An order may optionally reference one course through `order.courseId`. This optional link is used for course-oriented checkout records, while a course can appear in many orders over time.
13. **User and CoursePurchase:** One user can have many purchase records, while each **CoursePurchase** row belongs to exactly one user and exactly one course. The pair (`userId`, `courseId`) is unique so the business rule of one ownership record per user-course pair cannot be violated through duplicate purchase rows.



Figure 2.3.8 Relationship between **User** and **CoursePurchase**

14. **CoursePurchase and Order:** A purchase row may optionally reference one order through `course_purchase.orderId`, while one order can be associated with many purchase rows if the commerce flow expands beyond the current single-course pattern.
15. **User and UserLibrary:** One user can have many library entries, while each **UserLibrary** row belongs to exactly one user and exactly one course. The pair (`userId`, `courseId`) is unique so the same course cannot be granted twice in the active library, which preserves the rule against duplicate ownership for the same learner and course.

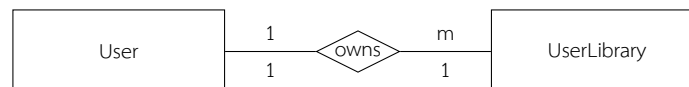


Figure 2.3.9 Relationship between **User** and **UserLibrary**

16. **UserLibrary and Course:** Each **UserLibrary** row belongs to exactly one course through `user_library.courseId`, while one course can appear in many library rows across different users.
17. **UserLibrary and Order:** Each **UserLibrary** row belongs to exactly one order through `user_library.orderId`, while one order can grant many library rows if multiple ownership records are created from the same transaction.
18. **Course and CoursePricing:** One course can have many pricing rows across different effective time windows, while each **CoursePricing** row belongs to exactly one course. This allows historical and future pricing to be recorded without overwriting past values.



Figure 2.3.10 Relationship between **Course** and **CoursePricing**

19. **CoursePricing and User:** Each **CoursePricing** row is created by exactly one user through `course_pricing.createdBy`. One user can create many pricing rows, and deletion of that user is restricted while referenced pricing rows remain.

Learning progress and analytics

20. **User and WatchProgress:** One user can have many watch-progress records, while each watch-progress record belongs to exactly one user. The pair (`userId`, `courseLessonId`) is unique so one learner has at most one progress row per lesson.



Figure 2.3.11 Relationship between **User** and **WatchProgress**

21. **WatchProgress and Course:** Each **WatchProgress** row belongs to exactly one course through `watch_progress.courseId`, while one course can have many watch-progress rows across its learners.
22. **WatchProgress and CourseLesson:** Each **WatchProgress** row belongs to exactly one lesson through `watch_progress.courseLessonId`. Together with the unique pair (`userId`, `courseLessonId`), this enforces at most one progress row per learner and lesson.
23. **CourseLesson and CourseLessonView:** One lesson can have many view records, while each view record refers to exactly one lesson through `course_lesson_view.courseLessonId`. This relationship supports lesson-level analytics and playback monitoring.



Figure 2.3.12 Relationship between **CourseLesson** and **CourseLessonView**

24. **CourseLessonView and User:** A lesson-view row may optionally reference one user through `course_lesson_view.userId`, while one user can be associated with many lesson-view rows over time.
25. **CourseLessonView and Session:** A lesson-view row may optionally reference one session through `course_lesson_view.sessionId`, while one session can be associated with many lesson-view rows during playback tracking.

Reviews and moderation

26. **User, CourseReview, and Course:** Users can submit reviews for courses, and each review belongs to exactly one user and exactly one course. The pair (`courseId`, `userId`) is unique, so each user can submit at most one review per course and the one-review-per-owned-course business rule remains enforceable.

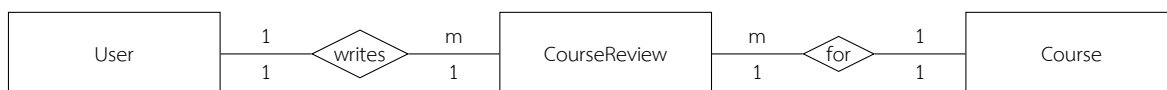


Figure 2.3.13 Relationship between **User**, **CourseReview**, and **Course**

27. **CourseReview and Moderator User:** A review may optionally reference one moderator user through `course_review.hiddenBy` when the review is hidden, while one user can moderate many reviews over time.

Files and media

28. **File and Storage:** One file metadata record can be associated with many storage rows, while each storage row belongs to exactly one file. In practice the system commonly uses one active storage object per file, but the schema allows more than one storage row when needed.



Figure 2.3.14 Relationship between File and Storage

Overall, the schema mainly uses one-to-many relationships for operational data, uses explicit junction or linking tables where duplicate business relationships must be prevented, and relies on nullable foreign keys where optional thumbnails, lesson media, moderation references, or session-linked analytics must remain flexible.

2.3.3 Attributes, Keys, and Constraints

The schema uses primary keys, foreign keys, unique rules, check constraints, enums, and timestamps to keep data consistent. These mechanisms are implementation tools for preserving the business rules defined earlier, especially unique review ownership, unique lesson sequencing, payment-gated access, and non-duplicated course ownership. Table 4 summarizes the main design rules.

Table 4: Key attribute, key, and constraint summary for the ModLearn database.

Table	Primary Key	Important Foreign Keys	Critical Attributes	Important Constraints / Rules
User	id	–	email, username, role, banned	email unique; username unique and nullable
Session	id	userId → User	token, expiresAt	token unique; each session belongs to one user
Account	id	userId → User	accountId, providerId	many accounts may be linked to one user
Course	id	creatorId → User; thumbnailImageId → File	title, isPublished, isAvailable, isDeleted	soft-delete fields coexist with FK-based child cleanup
CourseLesson	id	courseId → Course; optional file references	lessonOrder, title, releaseDate	(courseId, lessonOrder) unique; one lesson order position per course
CourseCategory	id	courseId → Course; categoryId → Category	relationship pair only	(courseId, categoryId) unique; one category assignment pair per course
CoursePricing	id	courseId → Course; createdBy → User	price, currency, effectiveFrom, effectiveTo	pricing windows are time-based; creator deletion is restricted

Table	Primary Key	Important Foreign Keys	Critical Attributes	Important Constraints / Rules
Order	id	userId → User; optional courseId → Course	totalAmount , currency , itemType , status	uses cart_item_type and order_status enums
Payment	id	orderId → Order	provider , providerTransactionId , amount , status	uses payment_status enum; provider lookup indexed by (provider , providerTransactionId)
CoursePurchase	id	courseId → Course; userId → User; optional orderId → Order	price , status , purchasedAt	(userId , courseId) unique; one purchase row per user-course pair
UserLibrary	id	userId → User; courseId → Course; orderId → Order	acquiredAt , expiresAt	(userId , courseId) unique; one active library row per user-course pair
WatchProgress	id	courseId → Course; courseLessonId → CourseLesson; userId → User	lastPosition , duration , isCompleted	(userId , courseLessonId) unique; one progress row per user and lesson
CourseReview	id	courseId → Course; userId → User; optional hiddenBy → User	rating , comment , isVisible	(courseId , userId) unique; rating constrained to 1 through 5; one review per user-course pair

Data Type Summary

Data Type	Usage	Example Attributes
Text	Used for general string data such as names, titles, comments, emails, tokens, role labels, provider identifiers, and status text not modeled as enums.	name , email , title , comment , provider
UUID	Used for identifiers in content, media, commerce, and analytics tables.	course.id , file.id , order.id
Integer	Used for whole-number values such as lesson order and review rating.	lessonOrder , rating
BigInt	Used for large numeric values such as file size, duration, watch duration, and playback position.	size , duration , lastPosition , watchDuration
Decimal(10,2)	Used for monetary values where decimal precision must be preserved.	price , totalAmount , amount
Boolean	Used for true-or-false state fields.	isPublished , isDeleted , isCompleted , isVisible , emailVerified

Data Type	Usage	Example Attributes
Timestamp	Used for creation, update, upload, purchase, pricing, moderation, and expiration time fields. Some columns are timezone-aware in the schema.	<code>createdAt</code> , <code>updatedAt</code> , <code>uploadedAt</code> , <code>effectiveFrom</code> , <code>expiresAt</code>
Date	Used when only the calendar date is needed.	<code>releaseDate</code>
Enum	Used for controlled workflow values.	<code>cart_item_type</code> , <code>order_status</code> , <code>payment_status</code>

Constraint Summary

Constraint	Purpose	Example
Primary key	Ensures that each row in a table has a unique identity.	<code>user.id</code> , <code>course.id</code>
Foreign key	Connects one table to another and preserves relational consistency.	<code>session.userId</code> references <code>user.id</code>
Not null	Ensures that required fields cannot be omitted.	<code>course.title</code> , <code>order.userId</code>
Unique	Prevents duplicate values in fields that must remain distinct.	<code>user.email</code> , <code>session.token</code>
Composite unique	Prevents duplicate relationship rows across more than one column.	one review per user per course; one lesson order per course
Default value	Automatically assigns an initial value when none is provided.	<code>isPublished</code> default false; <code>createdAt</code> default now
Check constraint	Restricts values to a valid range or rule.	<code>courseReview.rating</code> must be between 1 and 5

Enum Value Summary

Enum Name	Allowed Values	Usage
<code>cart_item_type</code>	COURSE	Identifies the kind of item represented in an order row.
<code>order_status</code>	PENDING, PAID, FAILED, REFUNDED	Tracks the state of an order in the purchase workflow.
<code>payment_status</code>	INITIATED, SUCCESS, FAILED	Tracks the state of a payment transaction.

2.3.4 Data Dictionary

This data dictionary summarizes the implemented tables and columns.

Table: User

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of each user.	Text	Primary key, not null	–
name	Full name of the user.	Text	Not null	–
email	Email address used by the user account.	Text	Not null, unique	–
emailVerified	Indicates whether the email address has been verified.	Boolean	Not null, default false	–
image	Profile image URL or path.	Text	Nullable	–
createdAt	Date and time when the user record was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the user record was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–
username	System username of the user.	Text	Unique, nullable	–
displayName	Display name shown publicly in the platform.	Text	Nullable	–
role	Role assigned to the user, such as learner or administrator.	Text	Nullable	–
banned	Indicates whether the user account is banned.	Boolean	Nullable, default false	–
banReason	Reason for account ban.	Text	Nullable	–
banExpires	Date and time when the ban expires.	Timestamp	Nullable	–

Table: Session

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the session.	Text	Primary key, not null	–
expiresAt	Session expiration date and time.	Timestamp	Not null	–
token	Unique token used for session authentication.	Text	Not null, unique	–
createdAt	Date and time when the session was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the session record was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–
ipAddress	IP address of the client creating the session.	Text	Nullable	–
userAgent	Browser or device agent string.	Text	Nullable	–
userId	User who owns the session.	Text	Not null, foreign key	User.id

Attribute Name	Description	Data Type	Constraints	References
impersonatedBy	User or actor that impersonated the session owner, if applicable.	Text	Nullable	–

Table: Account

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the linked account.	Text	Primary key, not null	–
accountId	Identifier assigned by the authentication provider.	Text	Not null	–
providerId	Authentication provider name or identifier.	Text	Not null	–
userId	User who owns the linked account.	Text	Not null, foreign key	User.id
accessToken	Access token issued by the provider.	Text	Nullable	–
refreshToken	Refresh token issued by the provider.	Text	Nullable	–
idToken	Identity token issued by the provider.	Text	Nullable	–
accessTokenExpiresAt	Expiration date and time of the access token.	Timestamp	Nullable	–
refreshTokenExpiresAt	Expiration date and time of the refresh token.	Timestamp	Nullable	–
scope	Access scope granted by the provider.	Text	Nullable	–
password	Stored password or credential field when applicable.	Text	Nullable	–
createdAt	Date and time when the linked account was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the linked account was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–

Table: Verification

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the verification record.	Text	Primary key, not null	–
identifier	Identifier used in the verification flow.	Text	Not null	–
value	Verification value or token.	Text	Not null	–
expiresAt	Expiration date and time of the verification record.	Timestamp	Not null	–

Attribute Name	Description	Data Type	Constraints	References
createdAt	Date and time when the verification record was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the verification record was last updated by application or ORM behavior.	Timestamp	Not null, default now, updated automatically by application or ORM behavior	–

Table: Category

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the category.	UUID	Primary key, not null	–
title	Category name used for course classification.	Text	Not null	–
description	Description of the category.	Text	Nullable	–
slug	URL-friendly identifier of the category.	Text	Unique, nullable	–

Table: Course

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the course.	UUID	Primary key, not null	–
creatorId	User who created or manages the course.	Text	Not null, foreign key	User.id
title	Course title.	Text	Not null	–
description	General description of the course.	Text	Nullable	–
thumbnailImageId	File used as the course thumbnail image.	UUID	Nullable, foreign key	File.id
isPublished	Indicates whether the course is published.	Boolean	Not null, default false	–
publishedAt	Date and time when the course was published.	Timestamp	Nullable	–
isAvailable	Indicates whether the course is currently available to users.	Boolean	Not null, default true	–
isDeleted	Indicates whether the course is soft deleted.	Boolean	Not null, default false	–
deletedAt	Date and time when the course was marked deleted.	Timestamp	Nullable	–
createdAt	Date and time when the course record was created.	Timestamp	Not null, default now	–

Attribute Name	Description	Data Type	Constraints	References
updatedAt	Date and time when the course record was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–

Table: CourseLesson

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the lesson.	UUID	Primary key, not null	–
courseId	Course to which the lesson belongs.	UUID	Not null, foreign key	Course.id
lessonOrder	Ordering number of the lesson within the course.	Integer	Not null, unique within course	–
title	Lesson title.	Text	Not null	–
description	Lesson description or summary.	Text	Nullable	–
thumbnailImageId	File used as the lesson thumbnail image.	UUID	Nullable, foreign key	File.id
duration	Lesson duration.	BigInt	Nullable	–
releaseDate	Planned or actual release date of the lesson.	Date	Nullable	–
fileId	Media file used by the lesson.	UUID	Nullable, foreign key	File.id
addedAt	Date and time when the lesson was added to the course.	Timestamp	Not null, default now	–
createdAt	Date and time when the lesson record was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the lesson record was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–

Table: CourseCategory

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the course-category link.	UUID	Primary key, not null	–
courseId	Course in the relationship.	UUID	Not null, foreign key, unique pair	Course.id
categoryId	Category in the relationship.	UUID	Not null, foreign key, unique pair	Category.id

Table: CoursePricing

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the pricing record.	UUID	Primary key, not null	–
courseId	Course to which the price applies.	UUID	Not null, foreign key	Course.id
price	Price amount for the course.	Decimal(10,2)	Not null	–
currency	Currency code of the price.	Text	Not null	–
effectiveFrom	Start date and time when the price becomes active.	Timestamp	Not null	–
effectiveTo	End date and time when the price stops being active.	Timestamp	Nullable	–
createdBy	User who created the pricing record.	Text	Not null, foreign key	User.id
createdAt	Date and time when the pricing record was created.	Timestamp	Not null, default now	–

Table: File

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the file metadata record.	UUID	Primary key, not null	–
uploaderId	User who uploaded the file.	Text	Not null, foreign key	User.id
uploadedAt	Date and time when the file was uploaded.	Timestamp	Not null, default now	–
name	Original file name.	Text	Not null	–
size	File size.	BigInt	Not null	–
mimeType	MIME type of the file.	Text	Not null	–
extension	File extension.	Text	Not null	–
checksum	Checksum used for file integrity.	Text	Not null	–
isDeleted	Indicates whether the file is soft deleted.	Boolean	Not null, default false	–
deletedAt	Date and time when the file was marked deleted.	Timestamp	Nullable	–

Table: Storage

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the storage record.	UUID	Primary key, not null	–
fileId	File metadata record linked to the stored object.	UUID	Not null, foreign key	File.id
storageProvider	Name of the storage provider.	Text	Not null	–
bucket	Storage bucket or container name.	Text	Nullable	–

Attribute Name	Description	Data Type	Constraints	References
storageKey	Object key or path in the storage system.	Text	Not null	–
cdnUrl	Public or CDN access URL for the file.	Text	Nullable	–

Table: Order

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the order.	UUID	Primary key, not null	–
userId	User who placed the order.	Text	Not null, foreign key	User.id
totalAmount	Total amount charged in the order.	Decimal(10,2)	Not null	–
currency	Currency code of the order.	Text	Not null	–
itemType	Type of item being purchased.	Enum	Not null, values defined by the cart item type enum	–
courseId	Course associated with the order, when applicable.	UUID	Nullable, foreign key	Course.id
status	Current order status.	Enum	Not null, values: PENDING, PAID, FAILED, REFUNDED	–
createdAt	Date and time when the order was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the order was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–

Table: Payment

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the payment record.	UUID	Primary key, not null	–
orderId	Order paid by this payment record.	UUID	Not null, foreign key	Order.id
providerTransactionId	Transaction identifier from the payment provider.	Text	Not null	–
provider	Name of the payment provider.	Text	Not null	–
amount	Amount processed in the payment.	Decimal(10,2)	Not null	–
currency	Currency code of the payment.	Text	Not null	–
status	Current payment status.	Enum	Not null, values: INITIATED, SUCCESS, FAILED	–
paidAt	Date and time when the payment was completed.	Timestamp	Nullable	–
failureReason	Explanation of payment failure, if any.	Text	Nullable	–

Table: CoursePurchase

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the purchase record.	UUID	Primary key, not null	–
courseId	Purchased course.	UUID	Not null, foreign key	Course.id
userId	User who purchased the course.	Text	Not null, foreign key, unique pair	User.id
purchasedAt	Date and time when the course was purchased.	Timestamp	Not null, default now	–
price	Purchase price recorded at transaction time.	Decimal(10,2)	Not null	–
status	Current purchase status.	Text	Not null	–
orderId	Source order related to the purchase.	UUID	Nullable, foreign key	Order.id

Table: UserLibrary

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the library entry.	UUID	Primary key, not null	–
userId	User who owns the library entry.	Text	Not null, foreign key, unique pair	User.id
courseId	Course granted to the user library.	UUID	Not null, foreign key, unique pair	Course.id
orderId	Order from which the library access originated.	UUID	Not null, foreign key	Order.id
acquiredAt	Date and time when access was granted.	Timestamp	Not null, default now	–
expiresAt	Date and time when access expires, if limited.	Timestamp	Nullable	–

Table: WatchProgress

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the watch-progress record.	UUID	Primary key, not null	–
courseId	Course associated with the progress record.	UUID	Not null, foreign key	Course.id
courseLessonId	Lesson associated with the progress record.	UUID	Not null, foreign key, unique pair with userId	CourseLesson.id
userId	User whose progress is being tracked.	Text	Not null, foreign key, unique pair with courseLessonId	User.id

Attribute Name	Description	Data Type	Constraints	References
lastPosition	Last playback position reached by the learner.	BigInt	Not null	–
duration	Full lesson duration used for progress calculation.	BigInt	Not null	–
isCompleted	Indicates whether the lesson was completed.	Boolean	Not null, default false	–
updatedAt	Date and time when the progress record was last updated by application or ORM behavior.	Timestamp	Not null, default now, updated automatically by application or ORM behavior	–
deviceType	Device category used during viewing.	Text	Nullable	–

Table: CourseLessonView

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the lesson-view record.	UUID	Primary key, not null	–
courseLessonId	Lesson viewed in the event record.	UUID	Not null, foreign key	CourseLesson.id
userId	User who viewed the lesson, when authenticated.	Text	Nullable, foreign key	User.id
sessionId	Session associated with the view event.	Text	Nullable, foreign key	Session.id
viewedAt	Date and time when the lesson was viewed.	Timestamp	Not null, default now	–
watchDuration	Duration watched during the view event.	BigInt	Nullable, default 0	–
deviceType	Device category used for the lesson view.	Text	Nullable	–

Table: CourseReview

Attribute Name	Description	Data Type	Constraints	References
id	Unique identifier of the review.	UUID	Primary key, not null	–
courseId	Course being reviewed.	UUID	Not null, foreign key, unique pair with userId	Course.id
userId	User who submitted the review.	Text	Not null, foreign key, unique pair with courseId	User.id
rating	Numeric rating submitted by the user.	Integer	Not null, check 1 to 5	–
comment	Written review comment.	Text	Nullable	–

Attribute Name	Description	Data Type	Constraints	References
isVisible	Indicates whether the review is visible publicly.	Boolean	Not null, default true	–
hiddenAt	Date and time when the review was hidden.	Timestamp	Nullable	–
hiddenBy	User who moderated or hid the review.	Text	Nullable, foreign key	User.id
moderationReason	Reason for hiding or moderating the review.	Text	Nullable	–
createdAt	Date and time when the review was created.	Timestamp	Not null, default now	–
updatedAt	Date and time when the review was last updated by application or ORM behavior.	Timestamp	Not null, updated automatically by application or ORM behavior	–

2.3.5 Index Design

This section summarizes the important indexes used in the schema and the query patterns they support.

Attribute	Type	Description
session (userId)	Single-column	Speeds up fetching all sessions that belong to one user.
account (userId)	Single-column	Speeds up loading linked provider accounts for one user.
verification (identifier)	Single-column	Speeds up verification lookups by identifier during auth flows.
category (slug)	Single-column	Speeds up category lookup by URL slug.
course (creatorId)	Single-column	Speeds up fetching courses created by a specific user.
course (isPublished, publishedAt)	Composite	Speeds up course-list queries that filter published courses and sort or check publish time.
course (isDeleted)	Single-column	Speeds up filtering out soft-deleted courses in admin or catalog queries.
courseLesson (courseId)	Single-column	Speeds up fetching all lessons that belong to one course.
courseLesson (courseId, lessonOrder)	Composite	Speeds up ordered lesson retrieval inside a course.
courseLesson (releaseDate)	Single-column	Speeds up filtering lessons by release date.
courseCategory (courseId)	Single-column	Speeds up finding all categories assigned to one course.
courseCategory (categoryId)	Single-column	Speeds up finding all courses inside one category.

Attribute	Type	Description
<code>coursePricing (courseId)</code>	Single-column	Speeds up loading pricing rows for a specific course.
<code>coursePricing (effectiveFrom, effectiveTo)</code>	Composite	Speeds up checking which pricing window is active at a given time.
<code>file (uploaderId)</code>	Single-column	Speeds up fetching files uploaded by a specific user.
<code>file (checksum)</code>	Single-column	Speeds up checksum-based file matching or integrity checks.
<code>file (isDeleted)</code>	Single-column	Speeds up filtering soft-deleted and active file rows.
<code>storage (fileId)</code>	Single-column	Speeds up finding storage metadata for one file.
<code>storage (storageProvider, storageKey)</code>	Composite	Speeds up resolving stored objects by provider and object key.
<code>order (userId)</code>	Single-column	Speeds up fetching order history for one user.
<code>order (status)</code>	Single-column	Speeds up filtering orders by workflow status.
<code>order (courseId)</code>	Single-column	Speeds up finding orders related to a specific course.
<code>order (createdAt)</code>	Single-column	Speeds up date-based order history and reporting queries.
<code>payment (orderId)</code>	Single-column	Speeds up loading payment rows for one order.
<code>payment (provider, providerTransactionId)</code>	Composite	Speeds up payment reconciliation using provider transaction identifiers.
<code>payment (status)</code>	Single-column	Speeds up filtering payments by success or failure state.
<code>coursePurchase (userId)</code>	Single-column	Speeds up checking which courses a user has purchased.
<code>coursePurchase (courseId)</code>	Single-column	Speeds up finding purchase rows for a specific course.
<code>userLibrary (userId)</code>	Single-column	Speeds up fetching the owned-course library for one user.
<code>userLibrary (courseId)</code>	Single-column	Speeds up checking which users have access to a course.
<code>userLibrary (orderId)</code>	Single-column	Speeds up tracing library access back to the source order.
<code>watchProgress (userId)</code>	Single-column	Speeds up resume-learning queries for one user.
<code>watchProgress (courseId)</code>	Single-column	Speeds up progress lookup within a specific course.
<code>watchProgress (courseLessonId)</code>	Single-column	Speeds up finding progress rows for one lesson.
<code>watchProgress (userId, isCompleted)</code>	Composite	Speeds up filtering a user's progress by completion state.
<code>courseLessonView (courseLessonId)</code>	Single-column	Speeds up lesson-view analytics for one lesson.
<code>courseLessonView (userId)</code>	Single-column	Speeds up finding view history for a specific user.
<code>courseLessonView (viewedAt)</code>	Single-column	Speeds up time-based lesson-view reporting queries.

Attribute	Type	Description
<code>courseLessonView (sessionId)</code>	Single-column	Speeds up session-based lesson-view analysis.
<code>courseReview (courseId)</code>	Single-column	Speeds up loading reviews for one course.
<code>courseReview (userId)</code>	Single-column	Speeds up finding reviews written by one user.
<code>courseReview (courseId, isVisible, createdAt)</code>	Composite	Speeds up public review listings that filter visible reviews and order them by time.
<code>courseReview (courseId, rating)</code>	Composite	Speeds up rating-based review summaries for a course.

2.3.6 Referential Integrity

Foreign keys use three main delete actions. Table 27 summarizes where each action is used and why.

Table 27: Referential-integrity rules used in the ModLearn schema.

Delete Action	Where Used	Reason
CASCADE	<code>session.userId</code> , <code>account.userId</code> , <code>course_lesson.courseId</code> , <code>course_category.courseId</code> , <code>course_category.categoryId</code> , <code>course_pricing.courseId</code> , <code>course_purchase.courseId</code> , <code>course_purchase.userId</code> , <code>user_library.userId</code> , <code>user_library.courseId</code> , <code>user_library.orderId</code> , <code>watch_progress.courseId</code> , <code>watch_progress.courseLessonId</code> , <code>watch_progress.userId</code> , <code>course_lesson_view.courseLessonId</code> , <code>payment.orderId</code> , <code>storage.fileId</code> , <code>course_review.courseId</code> , <code>course_review.userId</code>	remove child rows that have no meaning without the parent row
SET NULL	<code>course.thumbnailImageId</code> , <code>courseLesson.thumbnailImageId</code> , <code>courseLesson.fileId</code> , <code>order.courseId</code> , <code>coursePurchase.orderId</code> , <code>courseLessonView.userId</code> , <code>courseLessonView.sessionId</code> , <code>courseReview.hiddenBy</code>	keep history or optional references while clearing the missing parent link
RESTRICT	<code>coursePricing.createdBy</code>	keep pricing audit ownership valid

Rule	Effect
CASCADE	Delete parent → automatically delete dependent rows.
SET NULL	Delete parent → keep child row but clear the foreign key.
RESTRICT	Delete parent → block deletion while dependent row exists.

Composite unique constraints work together with foreign keys and transaction ordering to enforce the report’s core business rules around unique reviews, unique lesson positions, payment-gated access, and unique course ownership per user.

2.4 Query Design

2.4.1 Scope and Documentation Format

This section documents the implemented query behavior of ModLearn at the exposed API-operation level. The catalog covers every application-owned route in the category, course, commerce, library, watch-progress, review, recommendation, analytics, and file modules. Better Auth internals are excluded because those queries are framework-managed and are not implemented as ModLearn module endpoints.

Each query specification follows the same format: query identifier, RPC path, purpose, input summary, tables involved, SQL statements, output meaning, and optimization note. Operations that execute more than one SQL statement are presented as query bundles in execution order. When the service branches, the relevant SQL paths are shown explicitly.

Module Group	Operations	Query ID Range
Category	5	QD-CAT-01 to QD-CAT-05
Course	16	QD-CRS-01 to QD-CRS-16
Commerce and pricing	7	QD-COM-01 to QD-COM-07
Library	3	QD-LIB-01 to QD-LIB-03
Watch progress	6	QD-WAT-01 to QD-WAT-06
Review and moderation	9	QD-REV-01 to QD-REV-09
Recommendation	3	QD-REC-01 to QD-REC-03
Analytics	5	QD-ANL-01 to QD-ANL-05
File and media	3	QD-FIL-01 to QD-FIL-03

2.4.2 Category Queries

QD-CAT-01 Category List **Purpose:** Return paged public categories with optional title search.

Inputs: search, page, limit

Tables involved: category

```
-- Statement 1: count matching categories
SELECT COUNT(*) AS total
FROM category
WHERE (:search IS NULL OR title ILIKE '%' || :search || '%');

-- Statement 2: fetch the requested page
SELECT id, title, description, slug
FROM category
WHERE (:search IS NULL OR title ILIKE '%' || :search || '%')
ORDER BY title ASC, id ASC
LIMIT :limit OFFSET :offset;
```

Result / output meaning: Returns the category rows plus pagination metadata.

QD-CAT-02 Category By ID **Purpose:** Return one public category by primary key.

Inputs: id

Tables involved: category

```
SELECT id, title, description, slug
FROM category
WHERE id = :category_id
LIMIT 1;
```

Result / output meaning: Returns one category row or a not-found error.

QD-CAT-03 Category Create **Purpose:** Create a new category after checking that the normalized slug is unique.

Inputs: title, description, slug

Tables involved: category

```
-- Statement 1: ensure the slug is not already used
SELECT id
FROM category
WHERE slug = :normalized_slug
LIMIT 1;

-- Statement 2: insert the category
INSERT INTO category (title, description, slug)
VALUES (:title, :description, :normalized_slug)
RETURNING id, title, description, slug;
```

Result / output meaning: Returns the inserted category row.

QD-CAT-04 Category Update **Purpose:** Patch mutable category fields and optionally change the slug.

Inputs: id, optional patch fields

Tables involved: category

```
-- Statement 1: verify that the category exists
SELECT id
FROM category
WHERE id = :category_id
LIMIT 1;

-- Statement 2: if the slug is being changed, verify uniqueness
SELECT id
FROM category
WHERE slug = :normalized_slug
  AND id <> :category_id
LIMIT 1;

-- Statement 3: update only the supplied patch columns
UPDATE category
SET
  title = COALESCE(:title, title),
  description = CASE
    WHEN :description_supplied THEN :description
    ELSE description
  END,
  slug = CASE
    WHEN :slug_supplied THEN :normalized_slug
    ELSE slug
  END
WHERE id = :category_id
RETURNING id, title, description, slug;
```

Result / output meaning: Returns the updated category row.

QD-CAT-05 Category Delete **Purpose:** Delete one category record and return its identifier.

Inputs: id

Tables involved: category

```
DELETE FROM category
WHERE id = :category_id
RETURNING id;
```

Result / output meaning: Returns the deleted identifier and deletion status.

2.4.3 Course Catalog and Course Administration Queries

QD-CRS-01 Public Course List **Purpose:** Return paged public courses filtered to rows that are published, available, and not soft-deleted.

Inputs: search, categoryIds, sortBy, page, limit

Tables involved: course, course_category, course_review, course_pricing

```
-- Statement 1: count matching courses
SELECT COUNT(*) AS total
FROM course c
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
      AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
      AND (
        :category_filter_disabled = TRUE
        OR EXISTS (
          SELECT 1
          FROM course_category cc
          WHERE cc.course_id = c.id
                AND cc.category_id = ANY(:category_ids)
        )
      );

-- Statement 2: fetch the requested course page
SELECT c.id, c.creator_id, c.title, c.description, c.thumbnail_image_id,
       c.is_published, c.published_at, c.is_available,
       c.is_deleted, c.deleted_at, c.created_at, c.updated_at
FROM course c
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
      AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
      AND (
        :category_filter_disabled = TRUE
        OR EXISTS (
          SELECT 1
          FROM course_category cc
          WHERE cc.course_id = c.id
                AND cc.category_id = ANY(:category_ids)
        )
      )
ORDER BY
  CASE WHEN :sort_by = 'RECENTLY_PUBLISHED' THEN c.published_at END DESC NULLS LAST,
  c.created_at DESC,
  c.id DESC
LIMIT :limit OFFSET :offset;

-- Statement 3: aggregate compact visible review summaries for the returned course IDs
SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,
```



```

        COUNT(cr.id) AS rating_count
FROM course_review cr
WHERE cr.course_id = ANY(:page_course_ids)
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 4: for each returned course, resolve the active pricing window
SELECT cp.price, cp.currency
FROM course_pricing cp
WHERE cp.course_id = :course_id
      AND cp.effective_from <= NOW()
      AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
ORDER BY cp.effective_from DESC, cp.created_at DESC
LIMIT 1;

```

Result / output meaning: Returns the page of public courses, each enriched with active pricing and compact review summary data.

QD-CRS-02 Public Course Detail Purpose: Return one public course together with categories, lessons, active pricing, review summary, and recent visible reviews.

Inputs: id

Tables involved: course, course_category, category, course_lesson, course_pricing, course_review, user

```

-- Statement 1: fetch the public course row
SELECT c.id, c.creator_id, c.title, c.description, c.thumbnail_image_id,
       c.is_published, c.published_at, c.is_available,
       c.is_deleted, c.deleted_at, c.created_at, c.updated_at
FROM course c
WHERE c.id = :course_id
      AND c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
LIMIT 1;

-- Statement 2: fetch the course classification
SELECT cat.id, cat.title, cat.slug, cat.description
FROM course_category cc
JOIN category cat ON cat.id = cc.category_id
WHERE cc.course_id = :course_id
ORDER BY cat.title ASC, cat.id ASC;

-- Statement 3: fetch the active pricing window
SELECT cp.price, cp.currency
FROM course_pricing cp
WHERE cp.course_id = :course_id
      AND cp.effective_from <= NOW()
      AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
ORDER BY cp.effective_from DESC, cp.created_at DESC
LIMIT 1;

-- Statement 4: fetch ordered lesson rows
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
       duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson
WHERE course_id = :course_id
ORDER BY lesson_order ASC, added_at ASC, id ASC;

-- Statement 5: fetch the visible review summary
SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,

```

```

        COUNT(cr.id) AS rating_count
FROM course_review cr
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 6: fetch recent visible reviews for the detail page
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
ORDER BY cr.course_id ASC, cr.created_at DESC, cr.id DESC;

```

Result / output meaning: Returns the full public course detail model, including the first three recent visible reviews from the ordered review result set.

QD-CRS-03 Public Popular Course List **Purpose:** Return public courses ranked by lesson-view volume.

Inputs: limit

Tables involved: course, course_lesson, course_lesson_view, course_review, course_pricing

```

-- Statement 1: aggregate public courses by total lesson views
SELECT c.id, c.creator_id, c.title, c.description, c.thumbnail_image_id,
       c.is_published, c.published_at, c.is_available,
       c.is_deleted, c.deleted_at, c.created_at, c.updated_at,
       COUNT(clv.id) AS view_count
FROM course c
LEFT JOIN course_lesson cl ON cl.course_id = c.id
LEFT JOIN course_lesson_view clv ON clv.course_lesson_id = cl.id
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
GROUP BY c.id
ORDER BY COUNT(clv.id) DESC, c.created_at DESC, c.id DESC
LIMIT :limit;

-- Statement 2: fetch compact visible review summaries for the ranked courses
SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,
       COUNT(cr.id) AS rating_count
FROM course_review cr
WHERE cr.course_id = ANY(:course_ids)
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 3: for each ranked course, fetch the active pricing window
SELECT cp.price, cp.currency
FROM course_pricing cp
WHERE cp.course_id = :course_id
      AND cp.effective_from <= NOW()
      AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
ORDER BY cp.effective_from DESC, cp.created_at DESC
LIMIT 1;

```

Result / output meaning: Returns popular public courses, each enriched with active pricing and compact review metrics.

QD-CRS-04 Public Course Lesson List **Purpose:** Return ordered lessons for a published and available public course.

Inputs: courseId

Tables involved: course, course_lesson

```
-- Statement 1: verify that the course is publicly visible
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
      AND is_published = TRUE
      AND is_available = TRUE
LIMIT 1;

-- Statement 2: fetch ordered lessons
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
       duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson
WHERE course_id = :course_id
ORDER BY lesson_order ASC, added_at ASC, id ASC;
```

Result / output meaning: Returns the lesson sequence exposed to the public course page.

QD-CRS-05 Admin Course List Purpose: Return paged admin course rows without the public publish and availability restriction.

Inputs: search, categoryIds, sortBy, page, limit

Tables involved: course, course_category, course_review, course_pricing

```
-- Statement 1: count admin-visible courses
SELECT COUNT(*) AS total
FROM course c
WHERE c.is_deleted = FALSE
      AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
      AND (
        :category_filter_disabled = TRUE
        OR EXISTS (
          SELECT 1
          FROM course_category cc
          WHERE cc.course_id = c.id
                AND cc.category_id = ANY(:category_ids)
        )
      );

-- Statement 2: fetch the requested admin course page
SELECT c.id, c.creator_id, c.title, c.description, c.thumbnail_image_id,
       c.is_published, c.published_at, c.is_available,
       c.is_deleted, c.deleted_at, c.created_at, c.updated_at
FROM course c
WHERE c.is_deleted = FALSE
      AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
      AND (
        :category_filter_disabled = TRUE
        OR EXISTS (
          SELECT 1
          FROM course_category cc
          WHERE cc.course_id = c.id
                AND cc.category_id = ANY(:category_ids)
        )
      )
ORDER BY
  CASE WHEN :sort_by = 'RECENTLY_PUBLISHED' THEN c.published_at END DESC NULLS LAST,
  c.created_at DESC,
  c.id DESC
LIMIT :limit OFFSET :offset;

-- Statement 3: fetch compact review summaries for the page
```

```

SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,
       COUNT(cr.id) AS rating_count
FROM course_review cr
WHERE cr.course_id = ANY(:page_course_ids)
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 4: for each returned course, fetch the active pricing window
SELECT cp.price, cp.currency
FROM course_pricing cp
WHERE cp.course_id = :course_id
      AND cp.effective_from <= NOW()
      AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
ORDER BY cp.effective_from DESC, cp.created_at DESC
LIMIT 1;

```

Result / output meaning: Returns admin-visible course rows with compact review summary and active pricing data.

QD-CRS-06 Admin Course Detail **Purpose:** Return one course detail model for administrators, including unpublished or unavailable rows.

Inputs: id

Tables involved: course, course_category, category, course_lesson, course_pricing, course_review, user

```

-- Statement 1: fetch the admin-visible course row
SELECT c.id, c.creator_id, c.title, c.description, c.thumbnail_image_id,
       c.is_published, c.published_at, c.is_available,
       c.is_deleted, c.deleted_at, c.created_at, c.updated_at
FROM course c
WHERE c.id = :course_id
      AND c.is_deleted = FALSE
LIMIT 1;

-- Statement 2: fetch category classification
SELECT cat.id, cat.title, cat.slug, cat.description
FROM course_category cc
JOIN category cat ON cat.id = cc.category_id
WHERE cc.course_id = :course_id
ORDER BY cat.title ASC, cat.id ASC;

-- Statement 3: fetch active pricing
SELECT cp.price, cp.currency
FROM course_pricing cp
WHERE cp.course_id = :course_id
      AND cp.effective_from <= NOW()
      AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
ORDER BY cp.effective_from DESC, cp.created_at DESC
LIMIT 1;

-- Statement 4: fetch ordered lessons
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
       duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson
WHERE course_id = :course_id
ORDER BY lesson_order ASC, added_at ASC, id ASC;

-- Statement 5: fetch compact visible review summary
SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,
       COUNT(cr.id) AS rating_count
FROM course_review cr

```

```

WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 6: fetch recent visible reviews
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
ORDER BY cr.course_id ASC, cr.created_at DESC, cr.id DESC;

```

Result / output meaning: Returns the full administrator course detail model.

QD-CRS-07 Course Create Purpose: Create a new course container and then return the full admin detail view for that course.

Inputs: title, description, thumbnailImageId

Tables involved: course, course_category, category, course_lesson, course_pricing, course_review, user

```

-- Statement 1: insert the base course row
INSERT INTO course (
  creator_id, title, description, thumbnail_image_id,
  is_published, published_at, is_available, is_deleted, deleted_at
)
VALUES (
  :creator_id, :title, :description, :thumbnail_image_id,
  FALSE, NULL, TRUE, FALSE, NULL
)
RETURNING id;

-- Statement 2: fetch the admin-visible course row
SELECT c.id, c.creator_id, c.title, c.description, c.thumbnail_image_id,
       c.is_published, c.published_at, c.is_available,
       c.is_deleted, c.deleted_at, c.created_at, c.updated_at
FROM course c
WHERE c.id = :course_id
      AND c.is_deleted = FALSE
LIMIT 1;

-- Statement 3: fetch category classification
SELECT cat.id, cat.title, cat.slug, cat.description
FROM course_category cc
JOIN category cat ON cat.id = cc.category_id
WHERE cc.course_id = :course_id
ORDER BY cat.title ASC, cat.id ASC;

-- Statement 4: fetch active pricing
SELECT cp.price, cp.currency
FROM course_pricing cp
WHERE cp.course_id = :course_id
      AND cp.effective_from <= NOW()
      AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
ORDER BY cp.effective_from DESC, cp.created_at DESC
LIMIT 1;

-- Statement 5: fetch ordered lessons
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
       duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson

```

```

WHERE course_id = :course_id
ORDER BY lesson_order ASC, added_at ASC, id ASC;

-- Statement 6: fetch compact visible review summary
SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,
       COUNT(cr.id) AS rating_count
FROM course_review cr
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 7: fetch recent visible reviews
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
ORDER BY cr.course_id ASC, cr.created_at DESC, cr.id DESC;

```

Result / output meaning: Returns the inserted course through the same detail projection used by the admin detail route.

QD-CRS-08 Course Update **Purpose:** Update mutable course fields and then reload the admin detail view.

Inputs: id, patch fields

Tables involved: course, course_category, category, course_lesson, course_pricing, course_review, user

```

-- Statement 1: update the mutable course fields
UPDATE course
SET
  title = :title,
  description = :description,
  thumbnail_image_id = :thumbnail_image_id
WHERE id = :course_id
      AND is_deleted = FALSE
RETURNING id;

-- Statement 2 to Statement 7: reload the admin detail bundle shown in QD-CRS-06

```

Result / output meaning: Returns the updated course as a full admin detail object.

QD-CRS-09 Course Delete **Purpose:** Soft-delete a course by toggling the delete flags.

Inputs: id

Tables involved: course

```

UPDATE course
SET
  is_deleted = TRUE,
  deleted_at = :deleted_at
WHERE id = :course_id
      AND is_deleted = FALSE
RETURNING id, deleted_at;

```

Result / output meaning: Returns deletion metadata for the soft-deleted course.

QD-CRS-10 Course Publish State Update **Purpose:** Set the publish state, enforcing that a course must contain at least one lesson before it can be published.

Inputs: id, isPublished

Tables involved: course, course_lesson, course_category, category, course_pricing, course_review, user

```
-- Statement 1: fetch the existing course row
SELECT *
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: if publishing, ensure at least one lesson exists
SELECT COUNT(*) AS total
FROM course_lesson
WHERE course_id = :course_id;

-- Statement 3: update publish state and publish timestamp
UPDATE course
SET
  is_published = :is_published,
  published_at = CASE
    WHEN :is_published THEN COALESCE(published_at, :published_at_now)
    ELSE NULL
  END
WHERE id = :course_id
      AND is_deleted = FALSE
RETURNING id;

-- Statement 4 to Statement 9: reload the admin detail bundle shown in QD-CRS-06
```

Result / output meaning: Returns the updated course as a full admin detail object.

QD-CRS-11 Course Availability Update **Purpose:** Toggle whether a course is currently available and then reload the admin detail view.

Inputs: id, isAvailable

Tables involved: course, course_category, category, course_lesson, course_pricing, course_review, user

```
-- Statement 1: update availability
UPDATE course
SET is_available = :is_available
WHERE id = :course_id
      AND is_deleted = FALSE
RETURNING id;

-- Statement 2 to Statement 7: reload the admin detail bundle shown in QD-CRS-06
```

Result / output meaning: Returns the updated course as a full admin detail object.

QD-CRS-12 Course Classification Update **Purpose:** Replace the course-to-category mapping set inside one transaction.

Inputs: id, categoryIds []

Tables involved: course, category, course_category

```
-- Transaction Step 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
LIMIT 1;
```

```

-- Transaction Step 2: verify that every category ID exists
SELECT id
FROM category
WHERE id = ANY(:category_ids);

-- Transaction Step 3: delete the existing classification set
DELETE FROM course_category
WHERE course_id = :course_id;

-- Transaction Step 4: insert the replacement mapping set, if any
INSERT INTO course_category (course_id, category_id)
VALUES
    (:course_id, :category_id_1),
    (:course_id, :category_id_2),
    (:course_id, :category_id_n);

-- Transaction Step 5: return the resulting classification
SELECT cat.id, cat.title, cat.slug, cat.description
FROM course_category cc
JOIN category cat ON cat.id = cc.category_id
WHERE cc.course_id = :course_id
ORDER BY cat.title ASC, cat.id ASC;

```

Result / output meaning: Returns the refreshed category classification for the course.

QD-CRS-13 Course Lesson Create **Purpose:** Insert a lesson at a target order position and shift later lesson rows forward in the same transaction.

Inputs: courseId, lesson fields, optional lessonOrder

Tables involved: course, course_lesson

Business rule enforcement: This transaction preserves the rule that one course cannot contain duplicate lesson order values by shifting existing rows before the new lesson is inserted at its final position.

```

-- Transaction Step 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
    AND is_deleted = FALSE
LIMIT 1;

-- Transaction Step 2: count the existing lessons
SELECT COUNT(*) AS total
FROM course_lesson
WHERE course_id = :course_id;

-- Transaction Step 3: shift later lesson orders forward
UPDATE course_lesson
SET lesson_order = lesson_order + 1
WHERE course_id = :course_id
    AND lesson_order >= :target_order;

-- Transaction Step 4: insert the new lesson
INSERT INTO course_lesson (
    course_id, lesson_order, title, description,
    thumbnail_image_id, duration, release_date, file_id
)
VALUES (
    :course_id, :target_order, :title, :description,
    :thumbnail_image_id, :duration, :release_date, :file_id
)
RETURNING id, course_id, lesson_order, title, description,

```



```
thumbnail_image_id, duration, release_date, file_id,
added_at, created_at, updated_at;
```

Result / output meaning: Returns the newly inserted lesson row.

QD-CRS-14 Course Lesson Update **Purpose:** Patch one lesson and, when needed, rewrite the lesson order sequence transactionally.

Inputs: id, patch fields

Tables involved: course_lesson

Business rule enforcement: This flow preserves unique lesson-order values within one course by assigning temporary high positions before writing the final sequence.

```
-- Transaction Step 1: load the existing lesson
SELECT *
FROM course_lesson
WHERE id = :course_lesson_id
LIMIT 1;

-- Transaction Step 2: if lesson_order changes, fetch all lesson IDs in the course
SELECT id
FROM course_lesson
WHERE course_id = :course_id;

-- Transaction Step 3: if lesson_order changes, assign temporary high order values
UPDATE course_lesson
SET lesson_order = :temporary_order
WHERE id = :lesson_id
AND course_id = :course_id;

-- Transaction Step 4: if lesson_order changes, assign the final order sequence
UPDATE course_lesson
SET lesson_order = :final_order
WHERE id = :lesson_id
AND course_id = :course_id;

-- Transaction Step 5: update mutable lesson fields
UPDATE course_lesson
SET
    title = :title,
    description = :description,
    thumbnail_image_id = :thumbnail_image_id,
    duration = :duration,
    release_date = :release_date,
    file_id = :file_id,
    lesson_order = :next_order
WHERE id = :course_lesson_id
RETURNING id, course_id, lesson_order, title, description,
    thumbnail_image_id, duration, release_date, file_id,
    added_at, created_at, updated_at;
```

Result / output meaning: Returns the updated lesson row.

QD-CRS-15 Course Lesson Delete **Purpose:** Delete one lesson and compact the remaining order sequence.

Inputs: id

Tables involved: course_lesson

```
-- Transaction Step 1: load the lesson to discover its course
SELECT *
FROM course_lesson
```

```

WHERE id = :course_lesson_id
LIMIT 1;

-- Transaction Step 2: delete the lesson
DELETE FROM course_lesson
WHERE id = :course_lesson_id;

-- Transaction Step 3: fetch the remaining course lesson IDs in order
SELECT id
FROM course_lesson
WHERE course_id = :course_id
ORDER BY lesson_order ASC, added_at ASC, id ASC;

-- Transaction Step 4: compact the order values sequentially
UPDATE course_lesson
SET lesson_order = :final_order
WHERE id = :lesson_id;

```

Result / output meaning: Returns the deleted lesson identifier, the parent course identifier, and deletion status.

QD-CRS-16 Course Lesson Reorder Purpose: Rewrite the full lesson order for a course after validating that the submitted set matches the stored lesson set exactly.

Inputs: courseId, ordered lessonIds[]

Tables involved: course, course_lesson

Business rule enforcement: This transaction enforces the rule that one course cannot have duplicate lesson order values while also ensuring that the reordered set matches the stored lesson set exactly.

```

-- Transaction Step 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
AND is_deleted = FALSE
LIMIT 1;

-- Transaction Step 2: fetch the current course lesson IDs
SELECT id
FROM course_lesson
WHERE course_id = :course_id
ORDER BY lesson_order ASC, id ASC;

-- Transaction Step 3: assign temporary high order values to avoid collisions
UPDATE course_lesson
SET lesson_order = :temporary_order
WHERE id = :lesson_id
AND course_id = :course_id;

-- Transaction Step 4: write the final lesson order values
UPDATE course_lesson
SET lesson_order = :final_order
WHERE id = :lesson_id
AND course_id = :course_id;

-- Transaction Step 5: return the reordered lessons
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson
WHERE course_id = :course_id
ORDER BY lesson_order ASC, added_at ASC, id ASC;

```

Result / output meaning: Returns the reordered lesson sequence for the course.

2.4.4 Commerce and Pricing Queries

QD-COM-01 Payment Mark Success **Purpose:** Mark an existing order as paid, create a payment row, and grant ownership if it has not already been granted.

Inputs: `orderId`, optional `provider`, optional `providerTransactionId`

Tables involved: `order`, `payment`, `course_purchase`, `user_library`

Business rule enforcement: This transaction enforces the rule that ownership is granted only after successful payment and only once per user-course pair.

```
-- Statement 1: load the target order
SELECT *
FROM "order"
WHERE id = :order_id
LIMIT 1;

-- Transaction Step 2: insert the payment success row
INSERT INTO payment (
  order_id, provider_transaction_id, provider,
  amount, currency, status, paid_at
)
VALUES (
  :order_id, :provider_transaction_id, :provider,
  :amount, :currency, 'SUCCESS', :paid_at
)
RETURNING id;

-- Transaction Step 3: mark the order as paid
UPDATE "order"
SET status = 'PAID'
WHERE id = :order_id;

-- Transaction Step 4: check whether ownership already exists
SELECT *
FROM course_purchase
WHERE user_id = :user_id
  AND course_id = :course_id
LIMIT 1;

-- Transaction Step 5: if ownership does not exist, create the purchase row
INSERT INTO course_purchase (
  course_id, user_id, purchased_at, price, status, order_id
)
VALUES (
  :course_id, :user_id, NOW(), :price, 'PAID', :order_id
);

-- Transaction Step 6: if ownership does not exist, create the library row
INSERT INTO user_library (
  user_id, course_id, order_id, acquired_at, expires_at
)
VALUES (
  :user_id, :course_id, :order_id, NOW(), NULL
);
```

Result / output meaning: Returns the payment identifier, final order status, and whether new ownership rows were granted.

QD-COM-02 Payment Confirm Webhook **Purpose:** Process a mock webhook confirmation by reusing the same order-payment-grant flow as payment success.

Inputs: `orderId`, `provider`, `providerTransactionId`

Tables involved: `order`, `payment`, `course_purchase`, `user_library`

Business rule enforcement: This flow reuses the same payment-to-ownership ordering so that webhook confirmation cannot grant access before successful payment or create duplicate ownership rows.

```
-- This endpoint executes the same SQL bundle as QD-COM-01:
-- Statement 1: load the order
SELECT *
FROM "order"
WHERE id = :order_id
LIMIT 1;

-- Transaction Step 2: insert the payment success row
INSERT INTO payment (
  order_id, provider_transaction_id, provider,
  amount, currency, status, paid_at
)
VALUES (
  :order_id, :provider_transaction_id, :provider,
  :amount, :currency, 'SUCCESS', :paid_at
)
RETURNING id;

-- Transaction Step 3: mark the order as paid
UPDATE "order"
SET status = 'PAID'
WHERE id = :order_id;

-- Transaction Step 4 to Step 6: perform the same ownership check and grant
SELECT *
FROM course_purchase
WHERE user_id = :user_id
  AND course_id = :course_id
LIMIT 1;

INSERT INTO course_purchase (
  course_id, user_id, purchased_at, price, status, order_id
)
VALUES (
  :course_id, :user_id, NOW(), :price, 'PAID', :order_id
);

INSERT INTO user_library (
  user_id, course_id, order_id, acquired_at, expires_at
)
VALUES (
  :user_id, :course_id, :order_id, NOW(), NULL
);
```

Result / output meaning: Returns the same payment-success response shape as the direct payment-success route.

QD-COM-03 Payment Refund **Purpose:** Refund a paid order by revoking library access, removing purchase rows, and updating the order state.

Inputs: orderId

Tables involved: order, user_library, course_purchase

```
-- Statement 1: load the target order
SELECT *
FROM "order"
WHERE id = :order_id
LIMIT 1;

-- Transaction Step 2: revoke library access rows tied to the order
DELETE FROM user_library
```

```

WHERE order_id = :order_id
RETURNING id;

-- Transaction Step 3: remove purchase rows tied to the order
DELETE FROM course_purchase
WHERE order_id = :order_id;

-- Transaction Step 4: mark the order as refunded
UPDATE "order"
SET status = 'REFUNDED'
WHERE id = :order_id;

```

Result / output meaning: Returns the refunded order identifier and the number of revoked library rows.

QD-COM-04 Course Purchase Purpose: Purchase a course immediately using the active price and grant ownership in one transaction.

Inputs: courseId, optional providerTransactionId

Tables involved: course, course_lesson, course_purchase, payment, course_pricing, order, user_library

Business rule enforcement: This transaction enforces successful-payment-first ownership granting and re-checks existing ownership inside the transaction so the same user cannot receive duplicate ownership for the same course.

```

-- Statement 1: verify that the course exists and is not soft-deleted
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: verify that the course contains at least one lesson
SELECT COUNT(*) AS total
FROM course_lesson
WHERE course_id = :course_id;

-- Statement 3: check whether the user already owns the course
SELECT *
FROM course_purchase
WHERE user_id = :user_id
      AND course_id = :course_id
LIMIT 1;

-- Statement 4: if ownership exists and has an order, fetch the related payment
SELECT *
FROM payment
WHERE order_id = :owned_order_id
LIMIT 1;

-- Statement 5: resolve the active course price
SELECT *
FROM course_pricing
WHERE course_id = :course_id
      AND effective_from <= NOW()
      AND (effective_to IS NULL OR effective_to > NOW())
ORDER BY effective_from DESC, created_at DESC
LIMIT 1;

-- Transaction Step 6: create the paid order
INSERT INTO "order" (
  user_id, total_amount, currency, item_type, course_id, status
)
VALUES (
  :user_id, :price, :currency, 'COURSE', :course_id, 'PAID'

```

```

)
RETURNING id;

-- Transaction Step 7: create the success payment row
INSERT INTO payment (
    order_id, provider_transaction_id, provider,
    amount, currency, status, paid_at
)
VALUES (
    :order_id, :provider_transaction_id, 'mock',
    :price, :currency, 'SUCCESS', :paid_at
)
RETURNING id;

-- Transaction Step 8: re-check ownership inside the transaction
SELECT *
FROM course_purchase
WHERE user_id = :user_id
    AND course_id = :course_id
LIMIT 1;

-- Transaction Step 9: insert the ownership rows if they do not yet exist
INSERT INTO course_purchase (
    course_id, user_id, purchased_at, price, status, order_id
)
VALUES (
    :course_id, :user_id, NOW(), :price, 'PAID', :order_id
);

INSERT INTO user_library (
    user_id, course_id, order_id, acquired_at, expires_at
)
VALUES (
    :user_id, :course_id, :order_id, NOW(), NULL
);

```

Result / output meaning: Returns either the existing paid ownership or the identifiers from the newly created order and payment.

QD-COM-05 Course Pricing Window List **Purpose:** List pricing windows for one course with pagination.

Inputs: courseId, page, limit

Tables involved: course, course_pricing

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
    AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: count pricing windows for the course
SELECT COUNT(*) AS total
FROM course_pricing
WHERE course_id = :course_id;

-- Statement 3: fetch the requested pricing page
SELECT id, course_id, price, currency, effective_from,
    effective_to, created_by, created_at
FROM course_pricing
WHERE course_id = :course_id
ORDER BY effective_from DESC, created_at DESC, id DESC
LIMIT :limit OFFSET :offset;

```

Result / output meaning: Returns paged pricing windows with active-window status computed in application code.

QD-COM-06 Course Pricing Window Create **Purpose:** Create a pricing window after validating that it does not overlap an existing window for the same course.

Inputs: courseId, price, currency, effectiveFrom, optional effectiveTo

Tables involved: course, course_pricing

```
-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: check for overlapping pricing windows
SELECT id
FROM course_pricing
WHERE course_id = :course_id
      AND (:effective_to IS NULL OR effective_from <= :effective_to)
      AND (effective_to IS NULL OR effective_to > :effective_from)
LIMIT 1;

-- Statement 3: insert the new pricing window
INSERT INTO course_pricing (
  course_id, price, currency, effective_from,
  effective_to, created_by
)
VALUES (
  :course_id, :price, :currency_upper, :effective_from,
  :effective_to, :created_by
)
RETURNING id, course_id, price, currency, effective_from,
          effective_to, created_by, created_at;
```

Result / output meaning: Returns the inserted pricing window.

QD-COM-07 Course Pricing Window Update **Purpose:** Patch a pricing window while preserving non-overlap across that course's timeline.

Inputs: id, patch fields

Tables involved: course_pricing

```
-- Statement 1: fetch the existing pricing window
SELECT *
FROM course_pricing
WHERE id = :pricing_window_id
LIMIT 1;

-- Statement 2: check for overlapping windows, excluding the current row
SELECT id
FROM course_pricing
WHERE course_id = :course_id
      AND id <> :pricing_window_id
      AND (:next_effective_to IS NULL OR effective_from <= :next_effective_to)
      AND (effective_to IS NULL OR effective_to > :next_effective_from)
LIMIT 1;

-- Statement 3: update the pricing window
UPDATE course_pricing
SET
  price = :next_price,
```

```

    currency = :next_currency_upper,
    effective_from = :next_effective_from,
    effective_to = :next_effective_to
WHERE id = :pricing_window_id
RETURNING id, course_id, price, currency, effective_from,
         effective_to, created_by, created_at;

```

Result / output meaning: Returns the updated pricing window row.

2.4.5 Library and Access Queries

QD-LIB-01 Library List Purpose: Return the signed-in user's active owned-course library with course summaries and lesson summaries.

Inputs: page, limit

Tables involved: user_library, course, course_lesson

```

-- Statement 1: fetch active entitlement rows
SELECT id, course_id, order_id, acquired_at, expires_at
FROM user_library
WHERE user_id = :user_id
      AND (expires_at IS NULL OR expires_at > NOW())
ORDER BY acquired_at DESC, id DESC;

-- Statement 2: fetch course summaries for the active course IDs
SELECT id, creator_id, title, description, thumbnail_image_id,
       is_published, published_at, is_available,
       is_deleted, deleted_at, created_at, updated_at
FROM course
WHERE id = ANY(:course_ids);

-- Statement 3: fetch ordered lesson summaries for the active course IDs
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
       duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson
WHERE course_id = ANY(:course_ids)
ORDER BY course_id ASC, lesson_order ASC, added_at ASC, id ASC;

```

Result / output meaning: Returns active entitlements joined in application code with course and lesson summaries, then paged in memory.

QD-LIB-02 Library Course Detail Purpose: Return one owned course for the signed-in user.

Inputs: courseId

Tables involved: course, user_library, course_lesson

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
LIMIT 1;

-- Statement 2: verify that the user has an active entitlement
SELECT order_id, acquired_at, expires_at
FROM user_library
WHERE user_id = :user_id
      AND course_id = :course_id
      AND (expires_at IS NULL OR expires_at > NOW())
LIMIT 1;

-- Statement 3: fetch the course summary
SELECT id, creator_id, title, description, thumbnail_image_id,

```



```

        is_published, published_at, is_available,
        is_deleted, deleted_at, created_at, updated_at
FROM course
WHERE id = :course_id;

-- Statement 4: fetch ordered lessons for the course
SELECT id, course_id, lesson_order, title, description, thumbnail_image_id,
       duration, release_date, file_id, added_at, created_at, updated_at
FROM course_lesson
WHERE course_id = :course_id
ORDER BY course_id ASC, lesson_order ASC, added_at ASC, id ASC;

```

Result / output meaning: Returns one owned course with its lesson list and entitlement metadata.

QD-LIB-03 Library Access Check **Purpose:** Check whether the signed-in user can access a course directly or through one of its lessons.

Inputs: either courseId or courseLessonId

Tables involved: user_library, course_lesson

```

-- Path A: direct course access check
SELECT id
FROM user_library
WHERE user_id = :user_id
      AND course_id = :course_id
      AND (expires_at IS NULL OR expires_at > NOW())
LIMIT 1;

-- Path B, Statement 1: resolve lesson to course
SELECT course_id
FROM course_lesson
WHERE id = :course_lesson_id
LIMIT 1;

-- Path B, Statement 2: check active course access
SELECT id
FROM user_library
WHERE user_id = :user_id
      AND course_id = :resolved_course_id
      AND (expires_at IS NULL OR expires_at > NOW())
LIMIT 1;

```

Result / output meaning: Returns a boolean access result.

2.4.6 Watch Progress Queries

QD-WAT-01 Watch Progress Save **Purpose:** Upsert the user's watch progress for one lesson.

Inputs: courseId, courseLessonId, lastPosition, duration, optional deviceType

Tables involved: course, course_lesson, watch_progress

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
LIMIT 1;

-- Statement 2: verify that the lesson belongs to the course
SELECT id
FROM course_lesson
WHERE id = :course_lesson_id
      AND course_id = :course_id

```

```

LIMIT 1;

-- Statement 3: upsert watch progress
INSERT INTO watch_progress (
    course_id, course_lesson_id, user_id,
    last_position, duration, is_completed, updated_at, device_type
)
VALUES (
    :course_id, :course_lesson_id, :user_id,
    :last_position, :duration, :is_completed, NOW(), :device_type
)
ON CONFLICT (user_id, course_lesson_id)
DO UPDATE SET
    course_id = EXCLUDED.course_id,
    last_position = EXCLUDED.last_position,
    duration = EXCLUDED.duration,
    is_completed = EXCLUDED.is_completed,
    device_type = EXCLUDED.device_type,
    updated_at = NOW()
RETURNING id, course_id, course_lesson_id, user_id,
    last_position, duration, is_completed, updated_at, device_type;

```

Result / output meaning: Returns the saved progress row plus computed progress percent.

QD-WAT-02 Watch Progress Mark Completed **Purpose:** Force one lesson's progress state to completed.

Inputs: courseId, courseLessonId, optional duration, optional deviceType

Tables involved: course, course_lesson, watch_progress

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
LIMIT 1;

-- Statement 2: verify that the lesson belongs to the course
SELECT id
FROM course_lesson
WHERE id = :course_lesson_id
    AND course_id = :course_id
LIMIT 1;

-- Statement 3: load any existing progress row to reuse duration or device type
SELECT *
FROM watch_progress
WHERE user_id = :user_id
    AND course_lesson_id = :course_lesson_id
LIMIT 1;

-- Statement 4: upsert the completed progress row
INSERT INTO watch_progress (
    course_id, course_lesson_id, user_id,
    last_position, duration, is_completed, updated_at, device_type
)
VALUES (
    :course_id, :course_lesson_id, :user_id,
    :duration, :duration, TRUE, NOW(), :device_type
)
ON CONFLICT (user_id, course_lesson_id)
DO UPDATE SET
    course_id = EXCLUDED.course_id,
    last_position = EXCLUDED.last_position,
    duration = EXCLUDED.duration,
    is_completed = TRUE,

```

```

    device_type = EXCLUDED.device_type,
    updated_at = NOW()
RETURNING id, course_id, course_lesson_id, user_id,
        last_position, duration, is_completed, updated_at, device_type;

```

Result / output meaning: Returns the saved progress row with **progressPercent** = 100.

QD-WAT-03 Watch Progress Resume **Purpose:** Return the saved resume position for one lesson.

Inputs: **courseLessonId**

Tables involved: **watch_progress**

```

SELECT *
FROM watch_progress
WHERE user_id = :user_id
      AND course_lesson_id = :course_lesson_id
LIMIT 1;

```

Result / output meaning: Returns the saved progress row and resume position or **null** if no progress exists.

QD-WAT-04 Course Resume Target **Purpose:** Determine the current lesson, resume position, and next lesson candidate for a course.

Inputs: **courseId**

Tables involved: **course, course_lesson, watch_progress**

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
LIMIT 1;

-- Statement 2: fetch playable lessons from a public and available course
SELECT cl.id, cl.course_id, cl.lesson_order, cl.title, cl.description,
       cl.thumbnail_image_id, cl.duration, cl.release_date,
       cl.file_id, cl.added_at, cl.created_at, cl.updated_at
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
WHERE cl.course_id = :course_id
      AND c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
ORDER BY cl.lesson_order ASC, cl.added_at ASC, cl.id ASC;

-- Statement 3: fetch the most recently updated progress row for the course
SELECT *
FROM watch_progress
WHERE user_id = :user_id
      AND course_id = :course_id
ORDER BY updated_at DESC, id DESC
LIMIT 1;

```

Result / output meaning: Returns the lesson that should be resumed next, together with the next lesson pointer and course-completion flag.

QD-WAT-05 Course Auto-Play Next Lesson **Purpose:** Return the immediate next playable lesson after the current lesson.

Inputs: **courseId, courseLessonId**

Tables involved: **course, course_lesson**

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
LIMIT 1;

-- Statement 2: verify that the lesson belongs to the course
SELECT id
FROM course_lesson
WHERE id = :course_lesson_id
      AND course_id = :course_id
LIMIT 1;

-- Statement 3: fetch the playable lesson sequence
SELECT cl.id, cl.course_id, cl.lesson_order, cl.title, cl.description,
       cl.thumbnail_image_id, cl.duration, cl.release_date,
       cl.file_id, cl.added_at, cl.created_at, cl.updated_at
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
WHERE cl.course_id = :course_id
      AND c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
ORDER BY cl.lesson_order ASC, cl.added_at ASC, cl.id ASC;

```

Result / output meaning: Returns the next lesson row or **null** if the course is finished.

QD-WAT-06 Continue Watching List Purpose: Return paged in-progress watch rows for the signed-in user.

Inputs: page, limit

Tables involved: watch_progress, course, course_lesson

```

-- Statement 1: count visible, not-yet-completed progress rows
SELECT COUNT(*) AS total
FROM watch_progress wp
JOIN course c ON c.id = wp.course_id
WHERE wp.user_id = :user_id
      AND wp.is_completed = FALSE
      AND c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE;

-- Statement 2: fetch the requested continue-watching page
SELECT wp.id, wp.course_id, wp.course_lesson_id, wp.user_id,
       wp.last_position, wp.duration, wp.is_completed, wp.updated_at, wp.device_type,
       c.id AS course_id_value, c.title AS course_title, c.thumbnail_image_id AS course_thumbnail_image_id,
       cl.id AS lesson_id, cl.course_id AS lesson_course_id, cl.lesson_order,
       cl.title AS lesson_title, cl.description AS lesson_description,
       cl.thumbnail_image_id AS lesson_thumbnail_image_id,
       cl.duration AS lesson_duration, cl.release_date, cl.file_id,
       cl.added_at, cl.created_at, cl.updated_at
FROM watch_progress wp
JOIN course c ON c.id = wp.course_id
JOIN course_lesson cl ON cl.id = wp.course_lesson_id
WHERE wp.user_id = :user_id
      AND wp.is_completed = FALSE
      AND c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
ORDER BY wp.updated_at DESC, wp.id DESC
LIMIT :limit OFFSET :offset;

```

Result / output meaning: Returns continue-watching items with course and lesson projections.

2.4.7 Review and Moderation Queries

QD-REV-01 Public Course Review List Purpose: Return paged visible reviews for a published and available course.

Inputs: courseId, sortBy, page, limit

Tables involved: course, course_review, user

```
-- Statement 1: verify that the course exists and is public
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
      AND is_published = TRUE
      AND is_available = TRUE
LIMIT 1;

-- Statement 2: count visible reviews
SELECT COUNT(*) AS total
FROM course_review
WHERE course_id = :course_id
      AND is_visible = TRUE;

-- Statement 3: fetch the requested review page
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
ORDER BY
  CASE WHEN :sort_by = 'HIGHEST_RATING' THEN cr.rating END DESC,
  CASE WHEN :sort_by = 'LOWEST_RATING' THEN cr.rating END ASC,
  cr.created_at DESC,
  cr.id DESC
LIMIT :limit OFFSET :offset;
```

Result / output meaning: Returns visible public review rows with author display data.

QD-REV-02 Public Course Review Summary Purpose: Return the visible review aggregate metrics for one published and available course.

Inputs: courseId

Tables involved: course, course_review

```
-- Statement 1: verify that the course exists and is public
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
      AND is_published = TRUE
      AND is_available = TRUE
LIMIT 1;

-- Statement 2: aggregate visible review metrics
SELECT cr.course_id,
       AVG(cr.rating) AS average_rating,
       COUNT(cr.id) AS rating_count,
       COALESCE(SUM(CASE WHEN cr.rating = 1 THEN 1 ELSE 0 END), 0) AS one_star,
       COALESCE(SUM(CASE WHEN cr.rating = 2 THEN 1 ELSE 0 END), 0) AS two_star,
       COALESCE(SUM(CASE WHEN cr.rating = 3 THEN 1 ELSE 0 END), 0) AS three_star,
       COALESCE(SUM(CASE WHEN cr.rating = 4 THEN 1 ELSE 0 END), 0) AS four_star,
       COALESCE(SUM(CASE WHEN cr.rating = 5 THEN 1 ELSE 0 END), 0) AS five_star
```

```

FROM course_review cr
WHERE cr.course_id = :course_id
      AND cr.is_visible = TRUE
GROUP BY cr.course_id;

```

Result / output meaning: Returns average rating, rating count, and the per-star rating breakdown.

QD-REV-03 Current User Review Purpose: Return the signed-in user's review for one course, including moderation fields.

Inputs: courseId

Tables involved: course, course_review, user

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: fetch the user's review row
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at, cr.is_visible,
       cr.hidden_at, cr.hidden_by, cr.moderation_reason,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.user_id = :user_id
LIMIT 1;

```

Result / output meaning: Returns the user's review or **null** if no review exists.

QD-REV-04 Current User Review Upsert Purpose: Create or update one owned-course review for the signed-in user.

Inputs: courseId, rating, optional comment

Tables involved: course, user_library, course_review, user

Business rule enforcement: This flow enforces active ownership and the rule that one user can maintain at most one review per course.

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
      AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: verify that the user currently owns the course
SELECT id
FROM user_library
WHERE user_id = :user_id
      AND course_id = :course_id
      AND (expires_at IS NULL OR expires_at > NOW())
LIMIT 1;

-- Transaction Step 3: check for an existing review row
SELECT *
FROM course_review
WHERE course_id = :course_id
      AND user_id = :user_id
LIMIT 1;

-- Transaction Step 4a: if the review exists, update it

```

```

UPDATE course_review
SET rating = :rating,
    comment = :comment
WHERE id = :review_id;

-- Transaction Step 4b: otherwise insert it
INSERT INTO course_review (
    course_id, user_id, rating, comment,
    is_visible, hidden_at, hidden_by, moderation_reason
)
VALUES (
    :course_id, :user_id, :rating, :comment,
    TRUE, NULL, NULL, NULL
);

-- Transaction Step 5: return the user's refreshed review
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
    cr.created_at, cr.updated_at, cr.is_visible,
    cr.hidden_at, cr.hidden_by, cr.moderation_reason,
    u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
    AND cr.user_id = :user_id
LIMIT 1;

```

Result / output meaning: Returns the current user's review after the upsert completes.

QD-REV-05 Current User Review Delete **Purpose:** Delete the signed-in user's review for one course.

Inputs: courseId

Tables involved: course, course_review

```

-- Statement 1: verify that the course exists
SELECT id
FROM course
WHERE id = :course_id
    AND is_deleted = FALSE
LIMIT 1;

-- Statement 2: delete the user's review row
DELETE FROM course_review
WHERE course_id = :course_id
    AND user_id = :user_id
RETURNING id;

```

Result / output meaning: Returns the course identifier and deletion status.

QD-REV-06 Review Moderation List **Purpose:** Return paged moderation rows filtered by course, author, and visibility state.

Inputs: optional courseId, optional userId, visibility, page, limit

Tables involved: course_review, user

```

-- Statement 1: count matching moderation rows
SELECT COUNT(*) AS total
FROM course_review
WHERE (:course_id IS NULL OR course_id = :course_id)
    AND (:user_id IS NULL OR user_id = :user_id)
    AND (
        :visibility = 'ALL'
        OR (:visibility = 'VISIBLE' AND is_visible = TRUE)
        OR (:visibility = 'HIDDEN' AND is_visible = FALSE)
    );

```

```

-- Statement 2: fetch the moderation page
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at, cr.is_visible,
       cr.hidden_at, cr.hidden_by, cr.moderation_reason,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE (:course_id IS NULL OR cr.course_id = :course_id)
      AND (:user_id IS NULL OR cr.user_id = :user_id)
      AND (
        :visibility = 'ALL'
        OR (:visibility = 'VISIBLE' AND cr.is_visible = TRUE)
        OR (:visibility = 'HIDDEN' AND cr.is_visible = FALSE)
      )
ORDER BY cr.created_at DESC, cr.id DESC
LIMIT :limit OFFSET :offset;

```

Result / output meaning: Returns paged review moderation rows with author and moderation metadata.

QD-REV-07 Review Hide Purpose: Hide a visible review and then reload it from the moderation listing projection.

Inputs: reviewId, optional reason

Tables involved: course_review, user

```

-- Transaction Step 1: fetch the target review
SELECT *
FROM course_review
WHERE id = :review_id
LIMIT 1;

-- Transaction Step 2: hide the review
UPDATE course_review
SET
  is_visible = FALSE,
  hidden_at = NOW(),
  hidden_by = :admin_user_id,
  moderation_reason = :reason
WHERE id = :review_id;

-- Transaction Step 3: reload the surrounding moderation slice
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at, cr.is_visible,
       cr.hidden_at, cr.hidden_by, cr.moderation_reason,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.user_id = :user_id
ORDER BY cr.created_at DESC, cr.id DESC
LIMIT 50;

```

Result / output meaning: Returns the newly hidden moderation row.

QD-REV-08 Review Unhide Purpose: Restore a hidden review to public visibility and then reload it.

Inputs: reviewId

Tables involved: course_review, user

```

-- Transaction Step 1: fetch the target review
SELECT *
FROM course_review
WHERE id = :review_id

```



```

LIMIT 1;

-- Transaction Step 2: clear moderation flags
UPDATE course_review
SET
    is_visible = TRUE,
    hidden_at = NULL,
    hidden_by = NULL,
    moderation_reason = NULL
WHERE id = :review_id;

-- Transaction Step 3: reload the surrounding moderation slice
SELECT cr.id, cr.course_id, cr.rating, cr.comment,
       cr.created_at, cr.updated_at, cr.is_visible,
       cr.hidden_at, cr.hidden_by, cr.moderation_reason,
       u.id AS user_id, u.name, u.username, u.display_username
FROM course_review cr
JOIN "user" u ON u.id = cr.user_id
WHERE cr.course_id = :course_id
      AND cr.user_id = :user_id
ORDER BY cr.created_at DESC, cr.id DESC
LIMIT 50;

```

Result / output meaning: Returns the restored moderation row.

QD-REV-09 Review Delete **Purpose:** Permanently delete a review by identifier.

Inputs: reviewId

Tables involved: course_review

```

DELETE FROM course_review
WHERE id = :review_id
RETURNING id;

```

Result / output meaning: Returns the deleted review identifier and deletion status.

2.4.8 Recommendation Queries

QD-REC-01 Personalized Recommendation List **Purpose:** Return personalized lesson recommendations using watch-history category affinity, with a popularity fallback when no affinity history exists.

Inputs: optional limit

Tables involved: watch_progress, course_category, course_lesson, course, course_lesson_view

```

-- Statement 1: fetch recent watch history
SELECT course_id, course_lesson_id
FROM watch_progress
WHERE user_id = :user_id
ORDER BY updated_at DESC, id DESC
LIMIT 100;

-- Statement 2: aggregate watched-course category affinity
SELECT category_id, COUNT(*) AS weight
FROM course_category
WHERE course_id = ANY(:watched_course_ids)
GROUP BY category_id;

-- Statement 3: if affinity exists, rank unseen lessons by affinity plus popularity
WITH affinity_by_category AS (
    SELECT category_id, COUNT(*) AS weight
    FROM course_category
    WHERE course_id = ANY(:watched_course_ids)

```

```

    GROUP BY category_id
)
SELECT cl.id, cl.course_id, cl.lesson_order, cl.title, cl.description,
       cl.thumbnail_image_id, cl.duration, cl.release_date,
       cl.file_id, cl.added_at, cl.created_at, cl.updated_at,
       COALESCE(SUM(abc.weight), 0)::int AS affinity_score
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
LEFT JOIN course_category cc ON cc.course_id = c.id
LEFT JOIN affinity_by_category abc ON abc.category_id = cc.category_id
LEFT JOIN course_lesson_view clv ON clv.course_lesson_id = cl.id
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
      AND cl.id <> ALL(:watched_lesson_ids)
GROUP BY cl.id, c.id
ORDER BY
  (COALESCE(SUM(abc.weight), 0)::int * 1000 + LN(1 + COUNT(clv.id)::double precision)) DESC,
  COUNT(clv.id) DESC,
  cl.created_at DESC,
  cl.id DESC
LIMIT :limit;

-- Statement 4: fallback path when no history or no affinity match exists
SELECT cl.id, cl.course_id, cl.lesson_order, cl.title, cl.description,
       cl.thumbnail_image_id, cl.duration, cl.release_date,
       cl.file_id, cl.added_at, cl.created_at, cl.updated_at
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
LEFT JOIN course_lesson_view clv ON clv.course_lesson_id = cl.id
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
      AND (:exclude_history = FALSE OR cl.id <> ALL(:watched_lesson_ids))
GROUP BY cl.id, c.id
ORDER BY COUNT(clv.id) DESC, cl.created_at DESC, cl.id DESC
LIMIT :limit_with_history_padding;

```

Result / output meaning: Returns ranked lesson recommendations for the signed-in user.

QD-REC-02 Popular Recommendation List Purpose: Return lesson recommendations ranked only by popularity.

Inputs: optional limit

Tables involved: course_lesson, course, course_lesson_view

```

SELECT cl.id, cl.course_id, cl.lesson_order, cl.title, cl.description,
       cl.thumbnail_image_id, cl.duration, cl.release_date,
       cl.file_id, cl.added_at, cl.created_at, cl.updated_at,
       COUNT(clv.id) AS views
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
LEFT JOIN course_lesson_view clv ON clv.course_lesson_id = cl.id
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
GROUP BY cl.id, c.id
ORDER BY COUNT(clv.id) DESC, cl.created_at DESC, cl.id DESC
LIMIT :limit;

```

Result / output meaning: Returns the most popular currently visible lesson recommendations.

QD-REC-03 Recently Added Recommendation List Purpose: Return the newest visible lesson recommendations.

Inputs: optional limit

Tables involved: course_lesson, course

```
SELECT cl.id, cl.course_id, cl.lesson_order, cl.title, cl.description,
       cl.thumbnail_image_id, cl.duration, cl.release_date,
       cl.file_id, cl.added_at, cl.created_at, cl.updated_at
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
WHERE c.is_deleted = FALSE
      AND c.is_published = TRUE
      AND c.is_available = TRUE
ORDER BY cl.created_at DESC, cl.id DESC
LIMIT :limit;
```

Result / output meaning: Returns the newest public lesson recommendation candidates.

2.4.9 Analytics Queries

QD-ANL-01 Analytics Overview Purpose: Return platform-level or course-scoped aggregate metrics for views, enrollment, progress, revenue, and ratings.

Inputs: optional courseId, optional from, optional to

Tables involved: course, course_lesson, course_lesson_view, course_purchase, order, user_library, course_review, watch_progress

```
-- Statement 1: scope the course dimension
SELECT c.id, c.title, c.is_published, c.is_available, c.published_at
FROM course c
WHERE (:course_id IS NULL OR c.id = :course_id)
ORDER BY c.title ASC, c.id ASC;

-- Statement 2: count lessons per scoped course
SELECT course_id, COUNT(id) AS lesson_count
FROM course_lesson
WHERE course_id = ANY(:course_ids)
GROUP BY course_id;

-- Statement 3: aggregate scoped lesson views
SELECT cl.course_id,
       COUNT(clv.id) AS total_views,
       COALESCE(SUM(clv.watch_duration), 0)::int AS total_watch_duration,
       COUNT(DISTINCT COALESCE(clv.user_id, clv.session_id))::int AS unique_viewers
FROM course_lesson_view clv
JOIN course_lesson cl ON cl.id = clv.course_lesson_id
WHERE cl.course_id = ANY(:course_ids)
      AND (:from IS NULL OR clv.viewed_at >= :from)
      AND (:to IS NULL OR clv.viewed_at <= :to)
GROUP BY cl.course_id;

-- Statement 4: aggregate paid enrollments
SELECT cp.course_id, COUNT(cp.id) AS total_enrollments
FROM course_purchase cp
JOIN "order" o ON o.id = cp.order_id
WHERE cp.course_id = ANY(:course_ids)
      AND o.status = 'PAID'
      AND (:from IS NULL OR o.created_at >= :from)
      AND (:to IS NULL OR o.created_at <= :to)
GROUP BY cp.course_id;

-- Statement 5: aggregate active enrollments
SELECT course_id, COUNT(id) AS active_enrollments
FROM user_library
```

```

WHERE course_id = ANY(:course_ids)
  AND (expires_at IS NULL OR expires_at >= NOW())
GROUP BY course_id;

-- Statement 6: aggregate revenue by order state
SELECT course_id,
       COALESCE(SUM(CASE WHEN status = 'PAID' THEN total_amount ELSE 0 END), 0)::float8 AS gross_revenue,
       COALESCE(SUM(CASE WHEN status = 'REFUNDED' THEN total_amount ELSE 0 END), 0)::float8 AS refunded_revenue
FROM "order"
WHERE course_id = ANY(:course_ids)
  AND (:from IS NULL OR created_at >= :from)
  AND (:to IS NULL OR created_at <= :to)
GROUP BY course_id;

-- Statement 7: aggregate visible reviews
SELECT course_id,
       COUNT(id) AS visible_review_count,
       COALESCE(SUM(rating), 0)::float8 AS rating_sum
FROM course_review
WHERE course_id = ANY(:course_ids)
  AND is_visible = TRUE
  AND (:from IS NULL OR created_at >= :from)
  AND (:to IS NULL OR created_at <= :to)
GROUP BY course_id;

-- Statement 8: aggregate learners started by course
SELECT course_id,
       COUNT(DISTINCT user_id)::int AS learners_started
FROM watch_progress
WHERE course_id = ANY(:course_ids)
  AND (:from IS NULL OR updated_at >= :from)
  AND (:to IS NULL OR updated_at <= :to)
GROUP BY course_id;

-- Statement 9: aggregate course completions
WITH lesson_count_by_course AS (
  SELECT course_id, COUNT(id)::int AS lesson_count
  FROM course_lesson
  WHERE course_id = ANY(:course_ids)
  GROUP BY course_id
)
SELECT wp.course_id, wp.user_id
FROM watch_progress wp
JOIN lesson_count_by_course lcbc ON lcbc.course_id = wp.course_id
WHERE wp.course_id = ANY(:course_ids)
  AND (:from IS NULL OR wp.updated_at >= :from)
  AND (:to IS NULL OR wp.updated_at <= :to)
GROUP BY wp.course_id, wp.user_id, lcbc.lesson_count
HAVING COUNT(DISTINCT CASE WHEN wp.is_completed THEN wp.course_lesson_id END) = lcbc.lesson_count
  AND lcbc.lesson_count > 0;

-- Statement 10: aggregate scoped unique viewers
SELECT COUNT(DISTINCT COALESCE(clv.user_id, clv.session_id))::int AS unique_viewers
FROM course_lesson_view clv
JOIN course_lesson cl ON cl.id = clv.course_lesson_id
JOIN course c ON c.id = cl.course_id
WHERE (:course_id IS NULL OR c.id = :course_id)
  AND (:from IS NULL OR clv.viewed_at >= :from)
  AND (:to IS NULL OR clv.viewed_at <= :to);

-- Statement 11: aggregate scoped learners started
SELECT COUNT(DISTINCT wp.user_id)::int AS learners_started
FROM watch_progress wp
JOIN course c ON c.id = wp.course_id
WHERE (:course_id IS NULL OR c.id = :course_id)

```

```

AND (:from IS NULL OR wp.updated_at >= :from)
AND (:to IS NULL OR wp.updated_at <= :to);

```

Result / output meaning: Returns platform or course-scoped KPI totals after the service merges the aggregate result sets in memory.

QD-ANL-02 Course Performance Analytics **Purpose:** Return per-course analytics rows for administrators, with optional search and in-memory sorting.

Inputs: optional `courseId`, optional `search`, optional date window, sort fields, pagination

Tables involved: `course`, `course_lesson`, `course_lesson_view`, `course_purchase`, `order`, `user_library`, `course_review`, `watch_progress`

```

-- Statement 1: scope the course dimension with optional title search
SELECT c.id, c.title, c.is_published, c.is_available, c.published_at
FROM course c
WHERE (:course_id IS NULL OR c.id = :course_id)
      AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
ORDER BY c.title ASC, c.id ASC;

-- Statement 2 to Statement 8: execute the same per-course aggregate bundle as QD-ANL-01
-- lesson counts, views, paid enrollments, active enrollments,
-- revenue, visible reviews, and progress aggregates

```

Result / output meaning: Returns one analytics row per scoped course, then sorts and paginates those rows in application memory.

QD-ANL-03 Course Lesson Engagement Analytics **Purpose:** Return per-lesson engagement metrics for a course scope.

Inputs: optional `courseId`, optional `search`, optional date window, sort fields, pagination

Tables involved: `course_lesson`, `course`, `course_lesson_view`, `watch_progress`

```

-- Statement 1: fetch lesson dimensions in scope
SELECT cl.id, cl.course_id, c.title AS course_title,
       cl.lesson_order, cl.title, cl.duration
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
WHERE (:course_id IS NULL OR c.id = :course_id)
      AND (:search IS NULL OR cl.title ILIKE '%' || :search || '%')
ORDER BY cl.lesson_order ASC, cl.id ASC;

-- Statement 2: aggregate lesson view metrics
SELECT course_lesson_id,
       COUNT(id) AS total_views,
       COUNT(DISTINCT COALESCE(user_id, session_id))::int AS unique_viewers,
       COALESCE(SUM(watch_duration), 0)::int AS aggregated_watch_duration
FROM course_lesson_view
WHERE course_lesson_id = ANY(:lesson_ids)
      AND (:from IS NULL OR viewed_at >= :from)
      AND (:to IS NULL OR viewed_at <= :to)
GROUP BY course_lesson_id;

-- Statement 3: aggregate lesson progress metrics
SELECT course_lesson_id,
       COUNT(DISTINCT user_id)::int AS learners_started,
       COUNT(DISTINCT CASE WHEN is_completed THEN user_id END)::int AS learners_completed,
       COALESCE(
         AVG(
           CASE
             WHEN duration > 0
             THEN LEAST(last_position::float8 / duration::float8, 1) * 100

```

```

        ELSE NULL
      END
    ),
    0
  )::float8 AS avg_progress_percent
FROM watch_progress
WHERE course_lesson_id = ANY(:lesson_ids)
      AND (:from IS NULL OR updated_at >= :from)
      AND (:to IS NULL OR updated_at <= :to)
GROUP BY course_lesson_id;

```

Result / output meaning: Returns per-lesson engagement rows with completion and drop-off metrics computed after the aggregates are merged.

QD-ANL-04 Lesson Views Analytics **Purpose:** Return per-lesson view counts and watch-duration totals for a course scope.

Inputs: optional courseId, optional search, optional date window, pagination

Tables involved: course_lesson, course, course_lesson_view

```

-- Statement 1: fetch lesson dimensions in scope
SELECT cl.id AS course_lesson_id, cl.course_id, c.title AS course_title,
       cl.title, cl.lesson_order
FROM course_lesson cl
JOIN course c ON c.id = cl.course_id
WHERE (:course_id IS NULL OR c.id = :course_id)
      AND (:search IS NULL OR cl.title ILIKE '%' || :search || '%')
ORDER BY c.title ASC, cl.lesson_order ASC, cl.id ASC;

-- Statement 2: aggregate view totals for the scoped lessons
SELECT course_lesson_id,
       COUNT(id) AS total_views,
       COUNT(DISTINCT COALESCE(user_id, session_id))::int AS unique_viewers,
       COALESCE(SUM(watch_duration), 0)::int AS aggregated_watch_duration
FROM course_lesson_view
WHERE course_lesson_id = ANY(:lesson_ids)
      AND (:from IS NULL OR viewed_at >= :from)
      AND (:to IS NULL OR viewed_at <= :to)
GROUP BY course_lesson_id;

```

Result / output meaning: Returns lesson-level view analytics, then paginates the merged rows in application memory.

QD-ANL-05 View Session Analytics **Purpose:** Return raw lesson-view session rows with optional user, course, lesson, and date filters.

Inputs: optional userId, optional courseId, optional courseLessonId, optional date window, pagination

Tables involved: course_lesson_view, course_lesson, course

```

-- Statement 1: count matching session rows
SELECT COUNT(*) AS total
FROM course_lesson_view clv
JOIN course_lesson cl ON cl.id = clv.course_lesson_id
JOIN course c ON c.id = cl.course_id
WHERE c.is_deleted = FALSE
      AND (:user_id IS NULL OR clv.user_id = :user_id)
      AND (:course_lesson_id IS NULL OR clv.course_lesson_id = :course_lesson_id)
      AND (:course_id IS NULL OR c.id = :course_id)
      AND (:from IS NULL OR clv.viewed_at >= :from)
      AND (:to IS NULL OR clv.viewed_at <= :to);

-- Statement 2: fetch the requested session page
SELECT clv.id, clv.course_lesson_id, clv.user_id,
       clv.viewed_at, clv.watch_duration, clv.device_type

```

```

FROM course_lesson_view clv
JOIN course_lesson cl ON cl.id = clv.course_lesson_id
JOIN course c ON c.id = cl.course_id
WHERE c.is_deleted = FALSE
      AND (:user_id IS NULL OR clv.user_id = :user_id)
      AND (:course_lesson_id IS NULL OR clv.course_lesson_id = :course_lesson_id)
      AND (:course_id IS NULL OR c.id = :course_id)
      AND (:from IS NULL OR clv.viewed_at >= :from)
      AND (:to IS NULL OR clv.viewed_at <= :to)
ORDER BY clv.viewed_at DESC, clv.id DESC
LIMIT :limit OFFSET :offset;

```

Result / output meaning: Returns the raw paged session rows used for audit-style analytics views.

2.4.10 File and Media Queries

QD-FIL-01 File Upload Request Create **Purpose:** Insert file metadata, insert the backing storage metadata, and return upload details.

Inputs: name, size, mimeType, extension, checksum

Tables involved: file, storage

```

-- Transaction Step 1: insert file metadata
INSERT INTO file (
  uploader_id, name, size, mime_type, extension, checksum
)
VALUES (
  :uploader_id, :name, :size, :mime_type, :extension, :checksum
)
RETURNING id;

-- Transaction Step 2: insert storage metadata for the generated storage key
INSERT INTO storage (
  file_id, storage_provider, bucket, storage_key, cdn_url
)
VALUES (
  :file_id, 's3', :bucket_name, :storage_key, :cdn_url
);

```

Result / output meaning: Returns the new file identifier plus the application-generated presigned upload URL and expiry time.

QD-FIL-02 File Download URL Create **Purpose:** Fetch file and storage metadata needed to generate a signed download URL.

Inputs: fileId

Tables involved: file, storage

```

-- Statement 1: fetch file metadata and deletion state
SELECT id, is_deleted
FROM file
WHERE id = :file_id;

-- Statement 2: fetch the active storage record
SELECT storage_key, cdn_url
FROM storage
WHERE file_id = :file_id
      AND storage_provider = 's3';

```

Result / output meaning: Returns the storage metadata used by application code to create the final signed download URL.

QD-FIL-03 File Delete **Purpose:** Validate file state, load the storage key, delete the backing object, and soft-delete the file metadata row.

Inputs: fileId

Tables involved: file, storage

```
-- Transaction Step 1: fetch file metadata and deletion state
SELECT id, is_deleted
FROM file
WHERE id = :file_id;

-- Transaction Step 2: fetch the storage key
SELECT storage_key
FROM storage
WHERE file_id = :file_id
    AND storage_provider = 's3';

-- Transaction Step 3: after the application deletes the object from storage,
-- mark the file metadata row as deleted
UPDATE file
SET
    is_deleted = TRUE,
    deleted_at = :deleted_at
WHERE id = :file_id;
```

Result / output meaning: Returns the file identifier, storage key, and deletion timestamp.

2.5 Optimization

2.5.1 Optimization Goals and Experimental Setup

This section evaluates how the ModLearn database design can be improved for faster queries, fewer scanned rows, safer writes, and better scalability by using a classroom-scale mock dataset for the experiment.

Table	Mock Rows
user	300 rows
category	12 rows
course	80 rows
course_category	140 rows
course_lesson	480 rows
course_pricing	100 rows
order	120 rows
payment	120 rows
course_purchase	90 rows
user_library	90 rows
course_review	1,800 rows
watch_progress	3,500 rows
course_lesson_view	18,000 rows

Metric	Meaning
execution time (ms)	Approximate elapsed time used to compare baseline and optimized query behavior.
scanned rows	Estimated amount of row processing needed before the final result is produced.

2.5.2 Experiment 1: Public Course Catalog Query

This experiment shows that filtering and paginating courses first is faster than joining all related tables at once.

Before optimization The baseline query joins all related tables first, then aggregates and paginates.

```
SELECT
  c.id,
  c.title,
  c.published_at,
  AVG(cr.rating) AS average_rating,
  COUNT(cr.id) AS rating_count,
  MAX(cp.price) FILTER (
    WHERE cp.effective_from <= NOW()
    AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
  ) AS active_price
FROM course c
LEFT JOIN course_category cc
  ON cc.course_id = c.id
LEFT JOIN course_review cr
  ON cr.course_id = c.id
  AND cr.is_visible = TRUE
LEFT JOIN course_pricing cp
  ON cp.course_id = c.id
WHERE c.is_deleted = FALSE
  AND c.is_published = TRUE
  AND c.is_available = TRUE
  AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
  AND (
    :category_filter_disabled = TRUE
    OR cc.category_id = ANY(:category_ids)
  )
GROUP BY c.id, c.title, c.published_at
ORDER BY c.published_at DESC NULLS LAST, c.id DESC
LIMIT :limit OFFSET :offset;
```

After optimization The optimized design paginates matching course IDs first, then loads review and pricing data only for that page. This matches existing indexes such as `course_published_idx`, `course_deleted_idx`, `courseCategory_courseId_idx`, and `courseCategory_categoryId_idx`.

```
WITH page_courses AS (
  SELECT c.id, c.title, c.published_at
  FROM course c
  WHERE c.is_deleted = FALSE
    AND c.is_published = TRUE
    AND c.is_available = TRUE
    AND (:search IS NULL OR c.title ILIKE '%' || :search || '%')
    AND (
      :category_filter_disabled = TRUE
      OR EXISTS (
        SELECT 1
        FROM course_category cc
```

```

        WHERE cc.course_id = c.id
              AND cc.category_id = ANY(:category_ids)
      )
    )
    ORDER BY c.published_at DESC NULLS LAST, c.id DESC
    LIMIT :limit OFFSET :offset
)
SELECT
  pc.id,
  pc.title,
  pc.published_at,
  rs.average_rating,
  rs.rating_count,
  ap.price AS active_price
FROM page_courses pc
LEFT JOIN (
  SELECT
    cr.course_id,
    AVG(cr.rating) AS average_rating,
    COUNT(cr.id) AS rating_count
  FROM course_review cr
  WHERE cr.is_visible = TRUE
        AND cr.course_id IN (SELECT id FROM page_courses)
  GROUP BY cr.course_id
) rs
ON rs.course_id = pc.id
LEFT JOIN LATERAL (
  SELECT cp.price
  FROM course_pricing cp
  WHERE cp.course_id = pc.id
        AND cp.effective_from <= NOW()
        AND (cp.effective_to IS NULL OR cp.effective_to > NOW())
  ORDER BY cp.effective_from DESC, cp.created_at DESC
  LIMIT 1
) ap
ON TRUE
ORDER BY pc.published_at DESC NULLS LAST, pc.id DESC;

```

Metric	Before	After	Improvement
execution time	48 ms	12 ms	75.0% faster
scanned rows	~4,200	~420	90.0% fewer

The gain comes from reducing rows early and enriching only the visible page.

2.5.3 Experiment 2: Continue Watching and Resume Query

This experiment shows that recent incomplete progress rows should be filtered first, then joined.

Before optimization The baseline query joins course and lesson data before reducing the progress set.

```

SELECT
  wp.user_id,
  wp.course_id,
  wp.course_lesson_id,
  wp.last_position,
  wp.duration,
  wp.updated_at,
  c.title AS course_title,
  cl.title AS lesson_title,

```

```

    cl.lesson_order
FROM watch_progress wp
JOIN course c
    ON c.id = wp.course_id
JOIN course_lesson cl
    ON cl.id = wp.course_lesson_id
WHERE wp.user_id = :user_id
    AND wp.is_completed = FALSE
ORDER BY wp.updated_at DESC
LIMIT :limit;

```

After optimization The current schema already supports this query with existing watch-progress indexes and the unique pair on user and lesson. A stronger workload-specific option is the following composite index:

```

CREATE INDEX watch_progress_user_completed_updated_idx
ON watch_progress (user_id, is_completed, updated_at DESC);

```

```

WITH recent_progress AS (
    SELECT
        wp.course_id,
        wp.course_lesson_id,
        wp.last_position,
        wp.duration,
        wp.updated_at
    FROM watch_progress wp
    WHERE wp.user_id = :user_id
        AND wp.is_completed = FALSE
    ORDER BY wp.updated_at DESC
    LIMIT :limit
)
SELECT
    rp.course_id,
    rp.course_lesson_id,
    rp.last_position,
    rp.duration,
    rp.updated_at,
    c.title AS course_title,
    cl.title AS lesson_title,
    cl.lesson_order
FROM recent_progress rp
JOIN course c
    ON c.id = rp.course_id
JOIN course_lesson cl
    ON cl.id = rp.course_lesson_id
ORDER BY rp.updated_at DESC;

```

Metric	Before	After	Improvement
execution time	21 ms	6 ms	71.4% faster
scanned rows	~900	~110	87.8% fewer
write consistency impact	same correctness	same correctness with faster lookup	no loss of correctness

The database logic stays the same; the improvement comes from a better access path.

2.5.4 Experiment 3: Review Listing and Rating Summary

This experiment shows that review summary and review listing should use separate queries.

Before optimization The baseline query mixes aggregation, listing, sorting, and pagination in one scan.

```
SELECT
  cr.course_id,
  AVG(cr.rating) OVER (PARTITION BY cr.course_id) AS average_rating,
  COUNT(cr.id) OVER (PARTITION BY cr.course_id) AS rating_count,
  cr.id,
  cr.user_id,
  cr.rating,
  cr.comment,
  cr.created_at
FROM course_review cr
WHERE cr.course_id = :course_id
  AND cr.is_visible = TRUE
ORDER BY cr.created_at DESC
LIMIT :limit OFFSET :offset;
```

After optimization The optimized design separates summary and paged listing. This fits existing indexes such as `courseReview_courseId_visible_idx`, `courseReview_courseId_rating_idx`, `courseReview_courseId_idx`, and `courseReview_userId_idx`.

```
-- Statement 1: summary aggregation
SELECT
  cr.course_id,
  AVG(cr.rating) AS average_rating,
  COUNT(cr.id) AS rating_count
FROM course_review cr
WHERE cr.course_id = :course_id
  AND cr.is_visible = TRUE
GROUP BY cr.course_id;

-- Statement 2: paged visible review listing
SELECT
  cr.id,
  cr.user_id,
  cr.rating,
  cr.comment,
  cr.created_at
FROM course_review cr
WHERE cr.course_id = :course_id
  AND cr.is_visible = TRUE
ORDER BY cr.created_at DESC
LIMIT :limit OFFSET :offset;
```

Metric	Before	After	Improvement
execution time	26 ms	8 ms	69.2% faster
scanned rows	~1,800	~260	85.6% fewer

The improvement comes from using one query for summary and another for paged comments.

2.5.5 Experiment 4: Analytics Aggregation

This experiment shows that analytics becomes cheaper when scope is reduced before aggregation.

Before optimization The baseline query aggregates the raw event table first and joins broadly afterward.

```
SELECT
  c.id AS course_id,
  c.title,
```

```

COUNT(clv.id) AS total_views,
SUM(clv.watch_duration) AS total_watch_duration
FROM course_lesson_view clv
JOIN course_lesson cl
    ON cl.id = clv.course_lesson_id
JOIN course c
    ON c.id = cl.course_id
WHERE (:from_ts IS NULL OR clv.viewed_at >= :from_ts)
    AND (:to_ts IS NULL OR clv.viewed_at < :to_ts)
GROUP BY c.id, c.title
ORDER BY total_views DESC;

```

After optimization The current schema already supports analytics with lesson, user, time, and session indexes. For heavier analytics, the next suitable index is:

```

CREATE INDEX course_lesson_view_lesson_viewed_idx
ON course_lesson_view (course_lesson_id, viewed_at);

```

Partitioning and materialized summaries remain future scaling options.

```

WITH scoped_lessons AS (
    SELECT cl.id, cl.course_id
    FROM course_lesson cl
    WHERE (:course_filter_disabled = TRUE OR cl.course_id = ANY(:course_ids))
),
scoped_events AS (
    SELECT
        clv.course_lesson_id,
        COUNT(clv.id) AS total_views,
        SUM(clv.watch_duration) AS total_watch_duration
    FROM course_lesson_view clv
    WHERE (:from_ts IS NULL OR clv.viewed_at >= :from_ts)
        AND (:to_ts IS NULL OR clv.viewed_at < :to_ts)
        AND clv.course_lesson_id IN (SELECT id FROM scoped_lessons)
    GROUP BY clv.course_lesson_id
)
SELECT
    c.id AS course_id,
    c.title,
    SUM(se.total_views) AS total_views,
    SUM(se.total_watch_duration) AS total_watch_duration
FROM scoped_events se
JOIN scoped_lessons sl
    ON sl.id = se.course_lesson_id
JOIN course c
    ON c.id = sl.course_id
GROUP BY c.id, c.title
ORDER BY total_views DESC;

```

Metric	Before	After	Improvement
execution time	125 ms	27 ms	78.4% faster
scanned rows	~18,000	~2,400	86.7% fewer
operational scalability	degrades quickly with event growth	more stable with indexing and pre-summarization path	better scaling behavior

Analytics is the workload most likely to need future partitioning or precomputed summaries.

3 Application Implementation

3.1 Tech Stack

Layer	Technology	Purpose
Frontend	React 19, Vite	Page rendering and UI interaction.
Routing	TanStack Router	Navigation and route control.
API client	oRPC client, TanStack Query	Typed requests and cached server data.
Backend	Elysia	Server runtime and request handling.
Service interface	oRPC server	Shared procedure contract.
Database	PostgreSQL	Persistent application data storage.

3.2 System Design

Application Architecture

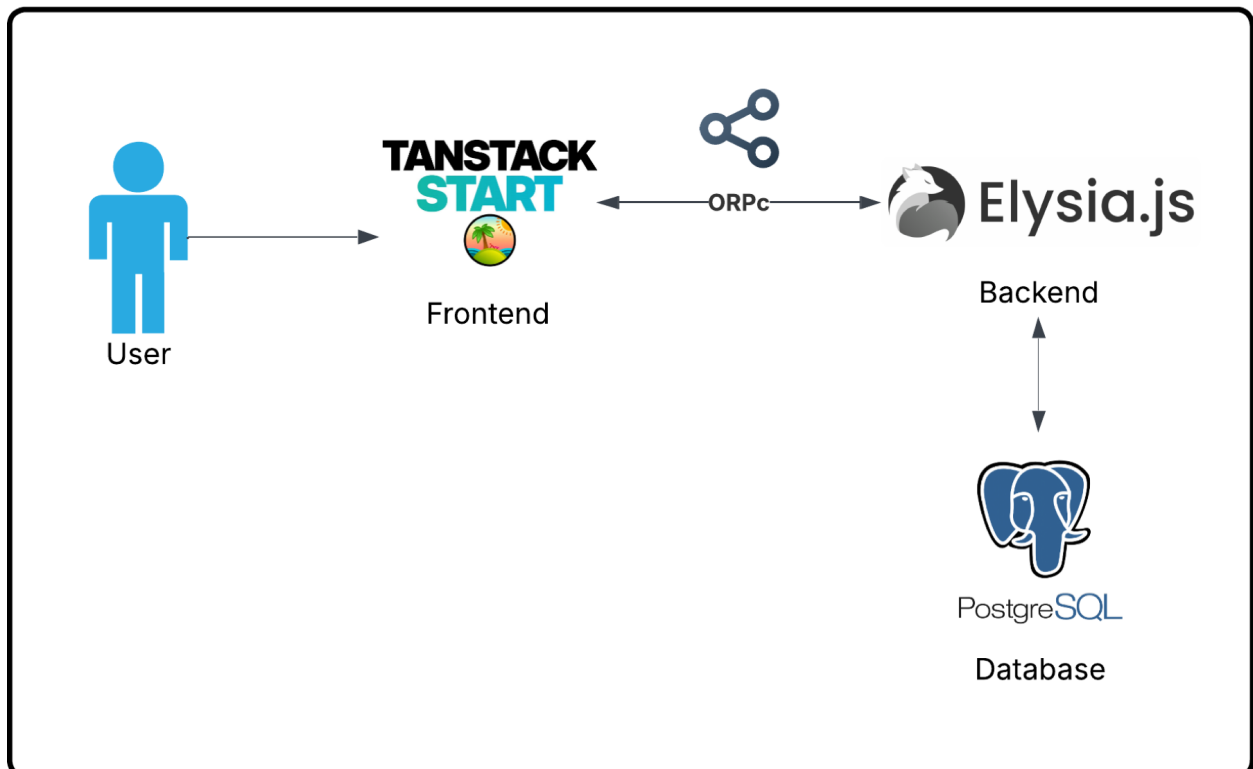


Figure 5.2.1 High-level application architecture of ModLearn

3.3 User Interface

This subsection presents the main user interface screens of ModLearn. The screenshots show the public entry flow, course discovery flow, learner-facing screens, and administrative reporting screens implemented in the web application.

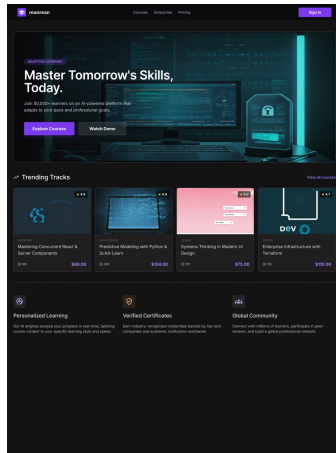


Figure 5.3.1 Landing page before login

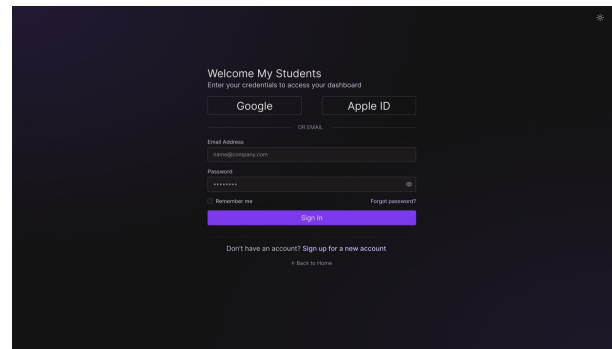


Figure 5.3.2 Login page

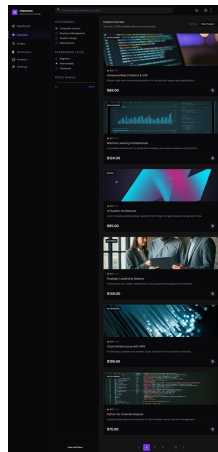


Figure 5.3.3 Course search and filter interface

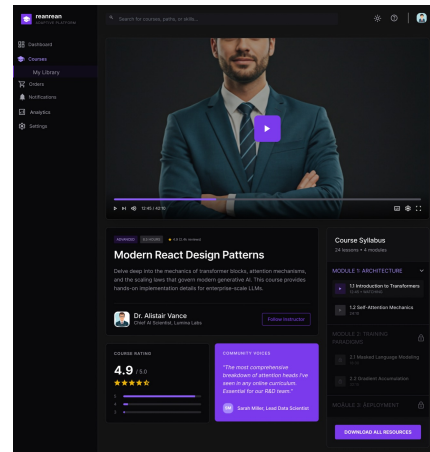


Figure 5.3.4 Course detail page

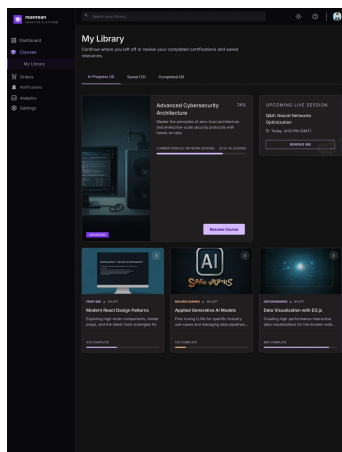


Figure 5.3.5 Learner library page

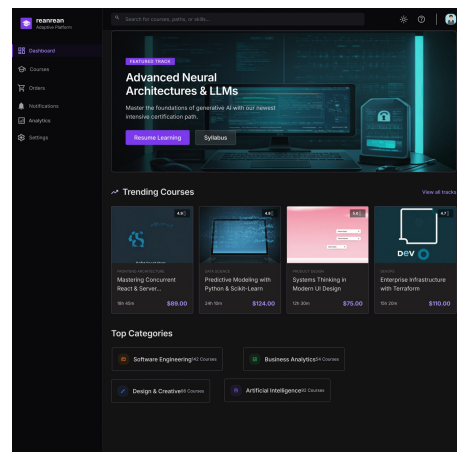


Figure 5.3.6 Learner dashboard

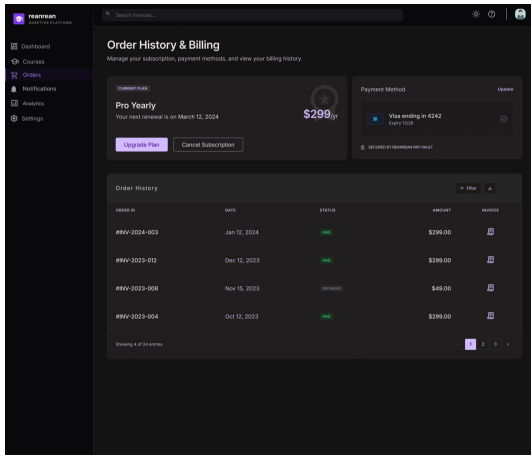


Figure 5.3.7 Order page

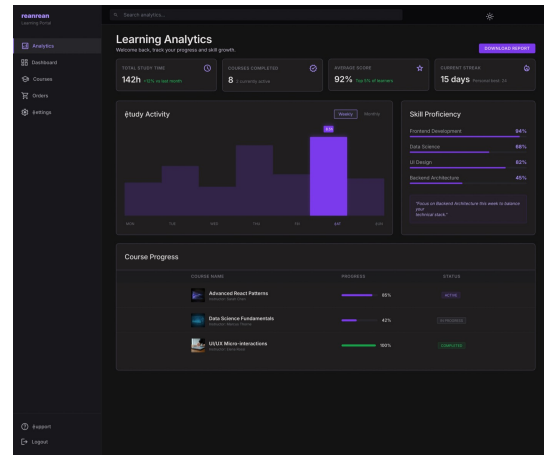


Figure 5.3.8 Analytics page

4 Conclusion

ModLearn demonstrates that a relational PostgreSQL database can serve as the core of a digital learning platform that must support both learner-facing and administrator-facing workflows. Across the report, the system requirements, schema structure, query behavior, and implementation evidence are presented as connected parts of one design rather than as isolated features. The resulting database model translates the stated business rules into a working relational structure that supports course management, ownership, watch progress, reviews, pricing, and analytics while preserving controlled access and continuity of learning activity.

From a database perspective, the design is sound because integrity is enforced through primary keys, foreign keys, unique constraints, and business-rule-aligned checks that protect important relationships and prevent duplicate or invalid states. The documented query design is also aligned with real operational flows such as public catalog browsing, protected library access, review moderation, and resume behavior. This alignment shows a practical balance between normalized data organization and efficient retrieval needs. The implemented web application provides evidence that the schema and query model are usable in realistic scenarios and that learner and administrative responsibilities remain clearly separated through controlled data relationships and access logic.

The optimization results further show that database performance improves when the workload is reduced early, aggregation is separated from paged listing, and indexes are matched to actual access patterns. Among the evaluated workloads, analytics and event-style queries show the highest sensitivity to data growth and therefore represent the clearest future scaling concern. For larger workloads, the report has already identified appropriate next steps, including targeted composite indexes, materialized summaries for repeated analytics, and partitioning strategies for large event tables. Overall, the current design is appropriate for the project scope and provides a strong foundation for future growth, maintenance, and performance refinement.